

Université Évry Paris-Saclay

Mention : Ingénierie des Systèmes Complexes

Parcours : M2 Robotique Industrielle

Master M2 Robotique Industrielle FI

Année universitaire
2025–2026

*Development of a Unified Digital Twin for Multi-Robot
Integration, Perception, and Neuro-Symbolic Planning*

Université de Toulouse

Maison de la Formation Jacqueline Auriol (MFJA)

NOM : IBRAHIM

Prénom : Ali

Numéro d'étudiant : 20255709

Statut : Formation initiale

Tuteurs de stage : Diane BURY et Olivier STASSE

Identification Page

Student

Name	Ali IBRAHIM
Programme	M2 ISC — Industrial Robotics track
Academic year	2025–2026
Student number	20255709
Telephone	07 50 79 35 89
Email	ali.ibrahim@etud.univ-evry.fr

Host organisation

Organisation	Université de Toulouse
Host service	Maison de la Formation Jacqueline Auriol (MFJA)
Legal address	118 route de Narbonne, 31062 Toulouse Cedex 9, France
Internship site	1 rue Tarfaya, 31400 Toulouse, France
Telephone	05 62 25 88 80

Internship supervisors

Diane BURY	Internship supervisor — MFJA; diane.bury@utoulouse.fr Telephone: 05 62 25 88 18
Olivier STASSE	Internship supervisor — LAAS-CNRS; olivier.stasse@laas.fr Telephone: 05 61 33 79 82

Academic supervisor

Name	Nicolas SEGUY
Role	Academic supervisor
Email	nicolas.seguy@univ-evry.fr
Telephone	01 69 47 75 51 (IBISC secretariat)

Mission

Period	2 March 2026 – 28 August 2026
Host unit	MFJA
Internship title	Développement d’une interface unifiée pour robots industriels et mobiles en simulation
Report language	English

Abstract

During this internship, I developed a unified Gazebo environment for the industrial and mobile robotics platforms of the MFJA. Room 315 combines two rail-transport cells, four industrial-robot workstations, cameras and programmable devices, but these heterogeneous elements needed compatible geometry, identities, interfaces and state descriptions before they could support repeatable experiments. The objective was therefore not only to reproduce the room visually, but to provide a coherent simulation for multi-robot launch, shuttle routing, scene observation and task-level English requests.

I reconstructed the MFJA third-floor geometry in Autodesk Inventor, designed the furniture and static contents of Rooms 314 and 315, and exported the resulting assets to Gazebo. In Room 315, I integrated the project-supplied transport parts and attributed upstream robot models, then implemented configuration-driven launch through Robot Operating System 2 (ROS 2), typed interfaces and a calibrated directed transport network. The rail backend manages stable shuttle identities, switches, stoppers, sensors, occupancy and temporary blocker relocation. This created reproducible operating scenes while keeping the different robot command interfaces explicit.

For task-level operation, I used learned models only to interpret uncertain inputs: a local language model converts English requests into checked task fields, and a paired-camera ResNet-18 estimator produces structured scene observations. PlanSys2 uses Planning Domain Definition Language (PDDL) rules to generate an inspectable symbolic action sequence. A software supervisor then authorises one typed action at a time and checks its observed effect before planning continues. This neuro-symbolic separation prevents either learned model from directly commanding simulated equipment.

The frozen visual model achieved 99.94% rail-segment classification accuracy on labelled Final Test targets. Under the stricter joint criterion, 90.90% had both the correct segment and a longitudinal error no greater than 5% of that segment's length. The isolated closed-loop campaign completed 12/12 declared tasks, and the five injected-fault scenarios passed 5/5 without a false success.

These results apply only to the evaluated Gazebo configuration, fixed synthetic cameras, known shuttle fleet and kinematic rail model. They do not establish transfer to physical cameras or machines, long-run reliability or machinery-safety certification.

Keywords: ROS 2; Gazebo Harmonic; industrial robotics; digital twin; multi-robot systems; flexible manufacturing; PDDL; PlanSys2; computer vision; language-and-vision interface; neuro-symbolic control; software supervision.

Résumé

Durant ce stage, j'ai développé, dans le simulateur robotique Gazebo, un environnement commun pour les robots industriels et mobiles de la Maison de la Formation Jacqueline Auriol (MFJA). La mission portait sur la salle 315, qui comprend deux cellules de transport sur rails et quatre postes de robots industriels. Le travail a combiné leur intégration via ROS 2, un système de transport multi-navettes calibré, la perception par caméras et l'interprétation de requêtes en anglais. Les modèles appris proposent des descriptions structurées de la tâche et de la scène, mais seuls les contrôles déterministes, la planification symbolique et la supervision autorisent les commandes. Cette interface neuro-symbolique n'est donc pas une politique vision-langage-action de bout en bout.

J'ai conçu sous Autodesk Inventor la géométrie du troisième étage, le mobilier et les équipements statiques des deux salles détaillées, puis je les ai exportés vers Gazebo. Pour la salle 315, j'ai créé son modèle dédié et assemblé ses postes et cellules sur rails avec les éléments de transport fournis par le projet et des modèles de robots externes cités.

Des lancements configurables et des interfaces ROS 2 typées relient les robots simulés, la topologie des rails et le superviseur d'exécution. Le modèle local Qwen2.5-1.5B-Instruct propose les champs structurés de la tâche ; un ResNet-18 à deux vues estime, pour chaque navette connue présente, son segment, sa position relative, sa boîte englobante et son état de chargement. Après validation, PlanSys2 et son planificateur POPF produisent une séquence symbolique exécutée action par action, avec vérification de chaque effet observé.

Le modèle visuel figé a atteint 99,94 % d'exactitude pour l'identification des segments sur les cibles annotées du test final. Selon le critère conjoint plus strict, 90,90 % des estimations associent le bon segment à une erreur longitudinale inférieure ou égale à 5 % de sa longueur. L'évaluation en boucle fermée dans Gazebo a réussi 12/12 scénarios positifs. La campagne d'injection de fautes a réussi 5/5 scénarios, sans faux succès. Ces résultats concernent exclusivement la configuration Gazebo évaluée et ne démontrent pas une aptitude au déploiement sur les équipements physiques.

Mots-clés : ROS 2 ; Gazebo ; robotique industrielle ; jumeau numérique ; systèmes multi-robots ; PDDL ; PlanSys2 ; vision ; VLA ; commande neuro-symbolique ; supervision logicielle.

Acknowledgements

I would like to sincerely thank Olivier Stasse and Diane Bury for their scientific guidance throughout this internship. Our weekly discussions and their constructive feedback helped me refine the technical architecture, improve the experimental methodology and develop the initial modelling assignment into a broader integration and research project for the MFJA platforms.

This work was granted access to the KAIROS high-performance computing resources operated by the CALMIP supercomputing centre.

I would also like to thank the teaching staff at my university for their guidance, availability and continued interest throughout the master's programme, including their support during the internship. I also thank the MFJA for providing the facilities and working environment in which this internship was carried out.

Use of Generative Artificial Intelligence

In accordance with the university's generative-AI guidance [42, p. 17], I declare that I used OpenAI ChatGPT [28] while preparing this report. I used it to improve language and document structure, suggest selected code refactorings, and assist with repetitive build and consistency checks. ChatGPT also assisted in drafting and iteratively reviewing parts of the initial Room 315 PDDL domain, especially repetitive predicate and action schemas.

ChatGPT did not define the objectives, select the evidence or approve the conclusions. Every suggestion was reviewed before it was accepted, adapted or rejected. Accepted textual and code changes were then checked by me. All reported results come from the recorded experimental artefacts and logs cited in this report, not from generated claims.

Glossary and Acronyms

Acronyms

AI	Artificial Intelligence
APE	Activité Principale Exercée, the French principal-activity code
API	Application Programming Interface
CAD	Computer-Aided Design
CLI	Command-Line Interface
CPU	Central Processing Unit
CSV	Comma-Separated Values
DZI	Project device-name prefix for an indexing-zone detector
F1	Harmonic mean of precision and recall
FI	Formation initiale, the non-apprenticeship study route used here
GUI	Graphical User Interface
GPU	Graphics Processing Unit
INSA	Institut National des Sciences Appliquées
INSEE	Institut National de la Statistique et des Études Économiques
ISAE	Institut Supérieur de l’Aéronautique et de l’Espace
ISC	Ingénierie des Systèmes Complexes
IoU	Intersection over Union, the overlap between predicted and reference regions divided by their union
JSON	JavaScript Object Notation
LAAS-CNRS	Laboratoire d’Analyse et d’Architecture des Systèmes, a laboratory of the Centre National de la Recherche Scientifique
LLM	Large Language Model
M2	Second year of a Master’s degree
MAE	Mean Absolute Error, the average absolute difference between a prediction and its reference value
MFJA	Maison de la Formation Jacqueline Auriol
ML	Machine Learning
NAF	Nomenclature d’Activités Française
PDDL	Planning Domain Definition Language
POPF	Partial Order Planning Forwards
QoS	Quality of Service
RGB	Red–Green–Blue colour representation
ROS	Robot Operating System
SDF	Simulation Description Format
STL	Stereolithography mesh format
TF	The time-indexed ROS hierarchy of coordinate-frame transforms
URDF	Unified Robot Description Format
VLA	Vision–Language–Action
VLM	Vision–Language Model
YAML	YAML Ain’t Markup Language

Project-specific terms

- Acceptance gate** A pass/fail criterion fixed before evaluation, distinct from a project-management decision milestone.
- Checkpoint** Model weights saved after an epoch; freezing prevents further updates before evaluation.
- Closed loop** Execution in which each command is followed by a new observation and a check of its expected effect before planning continues.
- Consumed historical Test** The predecessor test partition that was evaluated once and is therefore no longer locked or used as the current Final Test.
- Digital twin** A versioned digital representation of a defined physical system that includes its relevant information flows.
- Fail-closed** A policy that blocks an operation when required state is missing, stale, contradictory or from an unapproved source.
- Final Test** The preregistered post-freeze partition used as the project's final visual-model evidence.
- Hard-case Validation** The current development partition used for checkpoint selection and calibration, distinct from the historical Grouped Validation and the Final Test.
- Historical Grouped Validation** The predecessor validation partition that selected the checkpoint later used to initialise the V4 backbone; it is not a direct V4 input.
- Neuro-symbolic interface** An architecture in which learned components propose fixed-schema task and scene fields, while deterministic validation, symbolic planning and the software execution supervisor determine which commands may be published.
- ObservedState** A timestamped state snapshot whose accepted facts carry source, confidence and known/unknown/stale/conflicting status.
- Post-selection regression set** A small development partition checked only after checkpoint selection. Retained artefacts call it the *Development Canary* or use `canary` in paths and schema fields.
- Public topology** The common logical route graph and its canonical segment labels exposed to ROS 2 and planning, distinct from each rail's internal geometric curve identifiers.
- Selected Gazebo profile** A hash-identified Gazebo runtime that I selected after manually reviewing its retained evidence. Selection does not enable actuation; operator activation is separate.
- SHA-256** A cryptographic digest used here to identify the exact files or model weights associated with an evaluation.
- Shuttle** An independently controlled pallet carrier that travels around one of the two Room 315 rail cells and may be empty or carry a payload.
- Software execution supervisor** The deterministic component that checks state/safety before publishing a typed command, distinct from the human supervisors and operator.
- TaskGoal** The immutable structured task created after a fixed-schema text candidate passes rule validation and operator confirmation.

Contents

1	General Introduction	13
2	Host Organisation and MFJA Robotics Platforms	14
2.1	Université de Toulouse	14
2.2	Maison de la Formation Jacqueline Auriol	14
2.3	The Room 315 robotics environment	14
2.4	Organisation and responsibilities	15
2.5	Position of the mission within the host	16
2.6	Economic and strategic value	16
3	Mission, Project Management and Personal Contribution	17
3.1	Mission and work packages	17
3.2	Chronology and decision milestones	17
3.3	Requirements and review process	18
3.4	Personal responsibility, collaboration and resources	18
3.5	Autonomy and reviewed engineering decisions	19
3.6	Competencies and professional learning	20
4	Technical Background and Design Choices	21
4.1	ROS 2 as the integration layer	21
4.2	Gazebo Harmonic and description formats	21
4.3	Coordinate and rail representations	21
4.4	Symbolic task planning	22
4.5	Architectural alternatives and final design	22
5	Building the MFJA Digital Twin	24
5.1	CAD modelling and Gazebo integration strategy	24
5.2	World decomposition	25
5.3	Repository refactoring	25
5.4	Model integration	27
5.5	Launch profiles and reproducibility	30
5.6	Conclusion	30
6	Unified Multi-Robot Access and Control	31
6.1	What “unified” means	31
6.2	Multi-robot operation in one Gazebo process	31
6.3	Configuration-driven robot registry	31
6.4	Dynamic bridge generation	31
6.5	Joint-command helper	33
6.6	Conclusion	33
7	Room 315 Kinematic Transport System	34
7.1	Modelling decision: dynamic or kinematic	34
7.2	Kinematic transport backend	34
7.3	From SolidWorks points to a continuous rail graph	34
7.4	Topology, inherited switch names and segment vocabulary	36
7.5	Devices and their public identifiers	37
7.6	Multi-shuttle identity and lifecycle	37

7.7	Motion, slots and occupancy	37
7.8	Operator-facing observation and maintenance	38
7.9	Headway and collision constraints	38
7.10	Interior staging and dense blocker cases	38
7.11	Conclusion	38
8	Typed Interfaces, Contracts and Safety Supervision	39
8.1	Typed and versioned contracts	39
8.2	Accepted state and software supervision	39
8.3	Effect verification and emergency handling	41
8.4	Conclusion	41
9	AI Inputs: English and Visual-State Learning	42
9.1	Building the English task interface	42
9.2	Generating the Room 315 visual corpus	43
9.3	Designing and training the visual-state estimator	44
10	From an English Instruction to Shuttle Motion	48
10.1	Runtime architecture	48
10.2	Why the executive uses one-step replanning	49
10.3	Step 1—interpret and confirm the English task	49
10.4	Step 2—read the current scene and ground the goal	49
10.5	Step 3—build a fresh PDDL problem and plan	49
10.6	Steps 4 and 5—execute one step and verify its effect	49
10.7	Case B03 from instruction to arrival	51
11	Experiments and Results	52
11.1	Closed-loop campaign result	52
11.2	Final Test and runtime qualification	53
11.3	Software, language and full-floor checks	55
11.4	Review of the internship objectives	56
12	Limitations and Perspectives	57
12.1	Measured limits	57
12.2	Priority next steps	57
12.3	Continued use at MFJA	58
13	Conclusion	59
	Bibliography	60
A	Reproducibility Records	63
A.1	Identities and releases	63
A.2	Single verification procedure	64
A.3	Result-to-record index	64
B	Technical Reference and Detailed Matrices	65
B.1	Robot and rail configuration	65
B.2	Contracts and symbolic decisions	66
B.3	Visual-model lineage	66
B.4	Detailed experiment matrices	66

List of Figures

2.1	Gazebo overview of Room 315 and its four robot workstations.	15
2.2	Institutional setting, joint internship supervision and the intern's responsibilities. . .	15
3.1	Engineering, reporting and defence-preparation chronology.	18
4.1	Geometry and topology as complementary rail representations.	22
5.1	CAD-to-Gazebo progression from the third floor to Room 315.	26
5.2	ROS 2 package responsibilities and declared internal dependencies.	27
5.3	Physical-to-Gazebo comparison of the left-rail robot workstations.	28
5.4	Physical-to-Gazebo comparison of the right-rail robot workstations.	29
5.5	Physical-to-Gazebo comparison of TIAGo.	30
6.1	Configuration-driven multi-robot command and feedback isolation.	32
7.1	Physical-to-Gazebo comparison of the two Room 315 rail cells.	35
7.2	Ordered CSV source points and current Hermite rail geometry.	36
7.3	Paired Gazebo camera views of the complete eight-shuttle fleet.	38
8.1	Supervised sparse commands and controller feedback.	40
9.1	Paired Room 315 camera observation used for visual training.	43
9.2	Rail-isolated visual model and deterministic state derivation.	45
10.1	Room 315 English-to-motion runtime architecture.	50
11.1	Initial and final checkpoint-bound right-camera frames for case B03.	53

List of Tables

2.1	Stakeholders, operating needs and resulting design requirements.	16
3.1	Retained deliverable and review gate for each work package.	17
3.2	Professional interaction, engineering decisions and project continuity.	19
3.3	Engineering competencies and concrete project examples.	20
4.1	Representative architectures compared by the roles assigned to learning, planning and command execution.	23
5.1	Principal world and launch profiles.	25
6.1	Robot selectors and principal command surfaces.	32
6.2	Checks applied at the shared robot-access boundary.	33
7.1	Successor map for the public segment vocabulary shared by both rails.	37
9.1	V3 limitations and the corresponding V4 design responses.	44
9.2	Learned objectives and their role in the selected V4 training run.	46
9.3	Selected V4 optimisation, selection and evaluation controls.	46
9.4	Development comparison before and after the V4 redesign.	47
10.1	Principal decision cases in the Room 315 PDDL domain.	48
10.2	Stage-by-stage record for representative campaign case B03.	51
11.1	Coverage and outcome of the twelve-case closed-loop campaign.	53
11.2	Online runtime checks and offline labelled tolerances.	54
11.3	Global essentials from the one-shot Final Test.	54
11.4	Closed-loop fault cases, expected response and recorded outcome.	55
11.5	Assessment of the four internship objectives.	56
12.1	Priority experiments for extending the evidence.	58
A.1	Principal source, data and evidence identities.	63
A.2	Compact index of records supporting the reported results.	64
B.1	Operational configuration moved from the main chapters.	65
B.2	Principal closed-loop contracts and authorised producers.	66
B.3	Case-level detail for the closed-loop campaign.	66
B.4	Per-rail metrics from the one-shot Final Test.	67
B.5	Development and observation-only reference results.	67
B.6	Files and boundaries of the implemented symbolic-planning path.	67

Chapter 1

General Introduction

The MFJA robots, transport cells and cameras did not yet form a coherent simulation for reliable planning and experiments. The original assignment was to build the Room 315 environment and reusable launch foundation. Its early completion enabled Phase 2 on task-level English and paired-camera observations. PlanSys2 requests symbolic plans from rules written in the Planning Domain Definition Language (PDDL) [18]. PDDL defines the objects, state, actions, preconditions and effects [19]. Learned models supply checked task and scene descriptions; they never issue commands.

Problem and objectives. I used Autodesk Inventor for the computer-aided-design (CAD) geometry [2], Gazebo to execute the scenes [21], and ROS 2 to connect their software components [23].

Engineering problem. *How can the heterogeneous robots, rail cells and overhead-camera observations of Room 315 be integrated into a reproducible Gazebo environment that transforms constrained English requests into checked shuttle motion without giving learned models direct command authority?*

To answer this question, the project pursued four engineering objectives:

- O1.** design the third-floor geometry and furnished-room contents, then integrate the Room 315 robots and cells in a maintainable Gazebo environment with selectable launch and unified ROS 2 access;
- O2.** implement calibrated, directed multi-shuttle transport with device semantics, routing, reservations and separation constraints;
- O3.** define typed command and state interfaces, together with supervision rules for checking commands and confirming their effects; and
- O4.** design and evaluate a neuro-symbolic workflow that combines natural-language goal interpretation and paired-camera state estimation with PDDL/PlanSys2 planning and software-supervised execution.

O1–O4 progress from simulation to task-level interaction and are assessed in table 11.5.

Method. Development followed short build–test–review cycles. I checked each layer before integrating it, then froze the software, data and model versions used for evaluation (appendix A). Preserving consistent geometry, identity and state across layers was the main integration challenge. The delivered Gazebo system starts six isolated robot instances and coordinates eight shuttles. The frozen estimator reached 99.94% segment accuracy; 12/12 closed-loop tasks and 5/5 fault cases met their predefined outcomes (chapter 11).

Project demonstration videos

Project demonstrations are available in the [online demonstration-video folder](#). The videos provide a visual overview; the measured results and reproducibility records remain in chapter 11 and appendix A.

Report structure. Chapters 2–4 introduce the host, mission and design choices. Chapters 5–8 develop the simulation, robot, transport and interface foundations; Chapters 9 and 10 add the AI inputs and checked planning loop. Chapters 11–13 assess the evidence, limitations and contributions against O1–O4.

Chapter 2

Host Organisation and MFJA Robotics Platforms

The project took place in a shared university facility, so its users, equipment and operating constraints shaped the technical design. Understanding Université de Toulouse, MFJA, Room 315 and the division of responsibilities is therefore necessary to explain the mission’s educational, operational and strategic value.

2.1 Université de Toulouse

The internship was hosted by Université de Toulouse, 118 route de Narbonne, 31062 Toulouse Cedex 9, and carried out at the MFJA site, 1 rue Tarfaya, 31400 Toulouse. The university has operated as an experimental public institution since 1 January 2025 [40, p. 5]. Its Nomenclature d’activités française/Activité principale exercée (NAF/APE) code is 85.42Z, classified by the French national statistics institute (INSEE) as *higher education* [13, 14]. The same institutional document reports about 39,000 students and 4,200 staff across the university [40, p. 5]; the internship itself took place in the smaller MFJA environment.

2.2 Maison de la Formation Jacqueline Auriol

The MFJA is a shared engineering-education centre on the Toulouse Aerospace innovation campus. Its activities combine initial and continuing education, research, innovation and skills transfer. It brings together more than 1,500 students from 15 mechanical-engineering programmes; its official description also identifies aeronautics and space partnerships and an automation and robotics laboratory [41]. Sharing the equipment among these activities required configurable, multi-user software that remains maintainable beyond one developer; this constraint shaped the package and launch architecture in Chapters 5–6.

2.3 The Room 315 robotics environment

Room 315 contains two flexible transport cells: closed-loop rail systems whose independently controlled pallet carriers are termed *shuttles*. The cells share one public topology—the canonical segment names and connectivity exposed to ROS 2 and planning—but have distinct calibrated geometries, meaning that each rail follows its own measured spatial path. Each cell has four numbered shuttle slots. Four industrial-robot workstations surround the two cells. In the configuration used for this project, right-rail slots 1–2 are served by the Yaskawa HC10DT workstation and slots 3–4 by the Stäubli TX2-60L workstation. On the left rail, slots 1–2 are served by the Yaskawa HC10 workstation and slots 3–4 by the KUKA KR6 R900 sixx workstation. Each circuit contains outer and inner branches, four switching zones, indexing detectors and stoppers. Both rail networks are directed, so a shuttle can traverse each segment only in its defined forward direction. A switching zone diverts a shuttle between branches, an indexing detector reports its alignment or presence, and a directional stopper holds or releases an approaching shuttle. A shuttle may be empty or carry a payload.

The installation therefore combines mechanical modelling, industrial automation, robot communication, shared-resource planning, computer vision and task-level language in one platform. Figure 2.1 shows the simulated layout; Chapters 7–8 define the rail topology, device semantics and supervisory constraints.

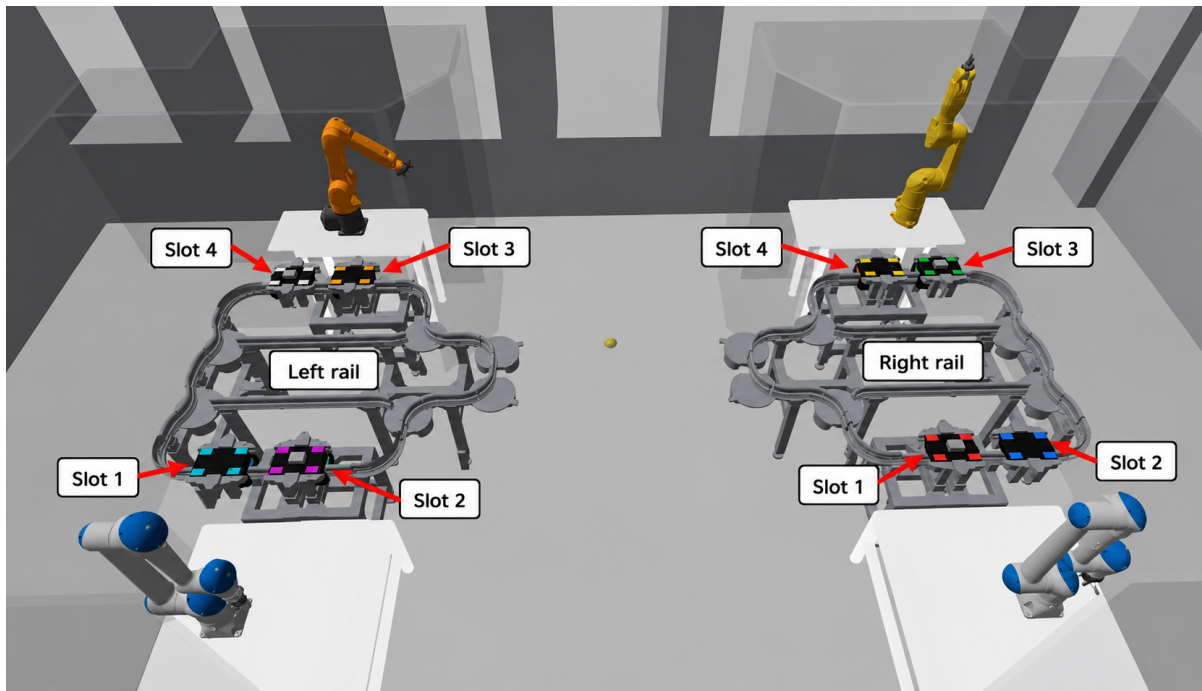


Figure 2.1: Gazebo view of Room 315. Four industrial-robot workstations surround two rail-transport cells containing eight stopped shuttles. The image identifies Slots 1–4 on the left and right rails. The measured rail geometries and device locations are presented in chapter 7.

2.4 Organisation and responsibilities

Figure 2.2 presents the supervision and responsibilities directly associated with the internship.

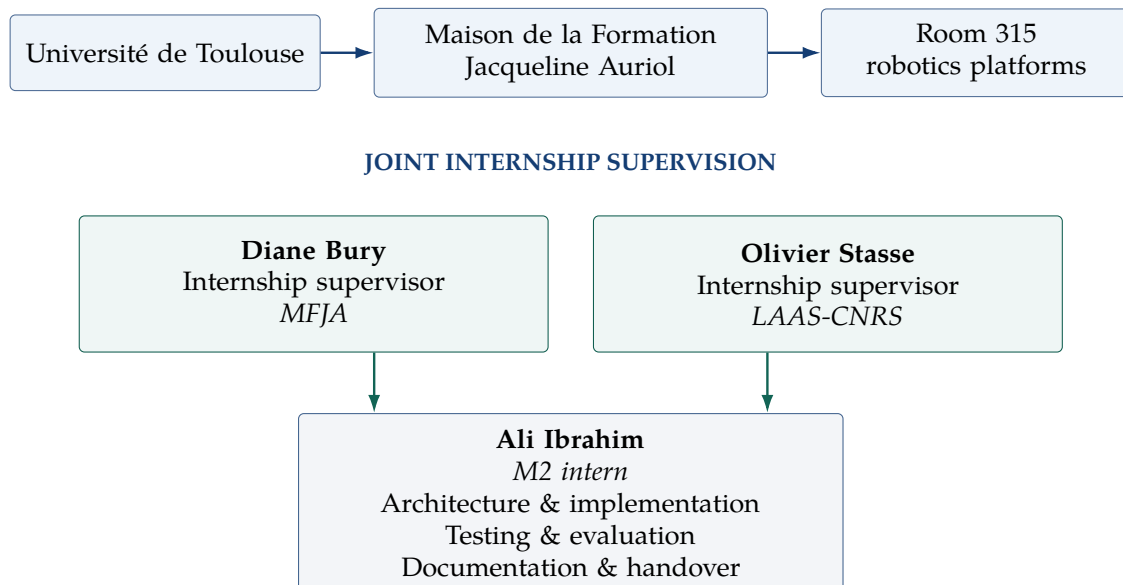


Figure 2.2: Institutional setting, joint internship supervision and the intern's responsibilities.

Diane Bury, affiliated with MFJA, and Olivier Stasse, affiliated with LAAS-CNRS, jointly supervised the internship. I was responsible for architecture, implementation, testing, evaluation, documentation and handover. Room 315 supports both teaching activities and research prototyping.

2.5 Position of the mission within the host

The platform has three main user groups, each with different operational needs.

Table 2.1: Stakeholders, operating needs and resulting design requirements.

Stakeholder	Operating need	Design consequence
MFJA platform staff	Prepare configurations, diagnose interfaces and maintain the environment	Versioned configuration, selectable profiles, diagnostics and operating guides
Students	Repeat ROS 2 and controller exercises without monopolising the physical cells	Focused worlds, isolated robots and predictable start-up
Researchers	Compare planning or perception under controlled conditions	Typed contracts, frozen case matrices and retained evidence

The environment is intended for continued use by MFJA students and researchers, including researchers from LAAS-CNRS. In teaching activities, simulation can serve as a preliminary validation stage before students proceed to supervised trials on the physical platform.

The resulting software and supporting documentation were handed over through the shared project repository to facilitate continued use and maintenance after the internship.

2.6 Economic and strategic value

Because the host is a public university, turnover and export revenue do not measure this mission's value. Its strategic document reports a 2024 global budget of nearly EUR 474 million and EUR 114 million in purchases [40, p. 5]; these university-wide figures indicate scale, not project return. The project is expected to create operational and educational value through:

- **reuse:** one model and launch framework avoids recreating an integration for each practical class or research project;
- **risk reduction:** simulation and supervision expose naming, topology and command errors before costly equipment is used;
- **asset visibility:** the digital environment supports remote preparation and makes available configurations easier to inspect;
- **training capacity:** repeatable scenarios create exercises and data without monopolising the physical cells;
- **maintainability:** typed interfaces, tests and documentation reduce dependence on knowledge held by one developer; and
- **technology transfer:** the platform may support demonstrations of current ROS 2, planning and AI methods while retaining explicit software-safety boundaries.

These constraints led me to prioritise configurable launch, stable interfaces and documentation that could be reused after the internship.

Chapter 3

Mission, Project Management and Personal Contribution

The internship was organised in two phases. The following sections separate the supervisors' input, reused resources and my own work. The chronology, review process and key architecture choices explain how the available resources became the delivered system.

3.1 Mission and work packages

My initial assignment was to model Room 315 and its robot and rail equipment in Gazebo. Gazebo simulates virtual robots, sensors and environments [21]. Finishing the assignment ahead of schedule enabled a second research work package without changing the internship dates.

The implemented work is organised into two phases:

Phase 1—simulation foundation Modelling Room 315 in Gazebo and creating reusable software that can start any selected combination of its simulated robots and rail equipment.

Phase 2—neuro-symbolic research Using paired-camera images to estimate equipment state, converting English requests into checked task goals, then planning and checking each action before execution.

Here, I use *neuro-symbolic* to mean learned image and language interpretation combined with rule-based planning expressed in PDDL [19] and integrated through PlanSys2 [18], followed by the execution checks I implemented. Phase 1 supplied the simulation needed by Phase 2; objectives O1–O4 in chapter 1 remain the units used to assess the results.

Table 3.1: Retained deliverable and review gate for each work package.

Phase	Retained deliverable	Gate used before the next stage
Phase 1	Selectable Gazebo scenes, six robot interfaces and a directed eight-shuttle rail runtime	Assets resolve, selected instances remain namespaced, rail routes and devices pass their component checks
Phase 2	Confirmed task and scene contracts, symbolic planning and checked one-step execution	Learned outputs have no command authority; plan translation is deterministic; every commanded effect is checked from feedback

3.2 Chronology and decision milestones

The internship schedule was fixed from the outset. As shown in figure 3.1, the technical phases overlap because integration, testing and correction continued after each main decision milestone.

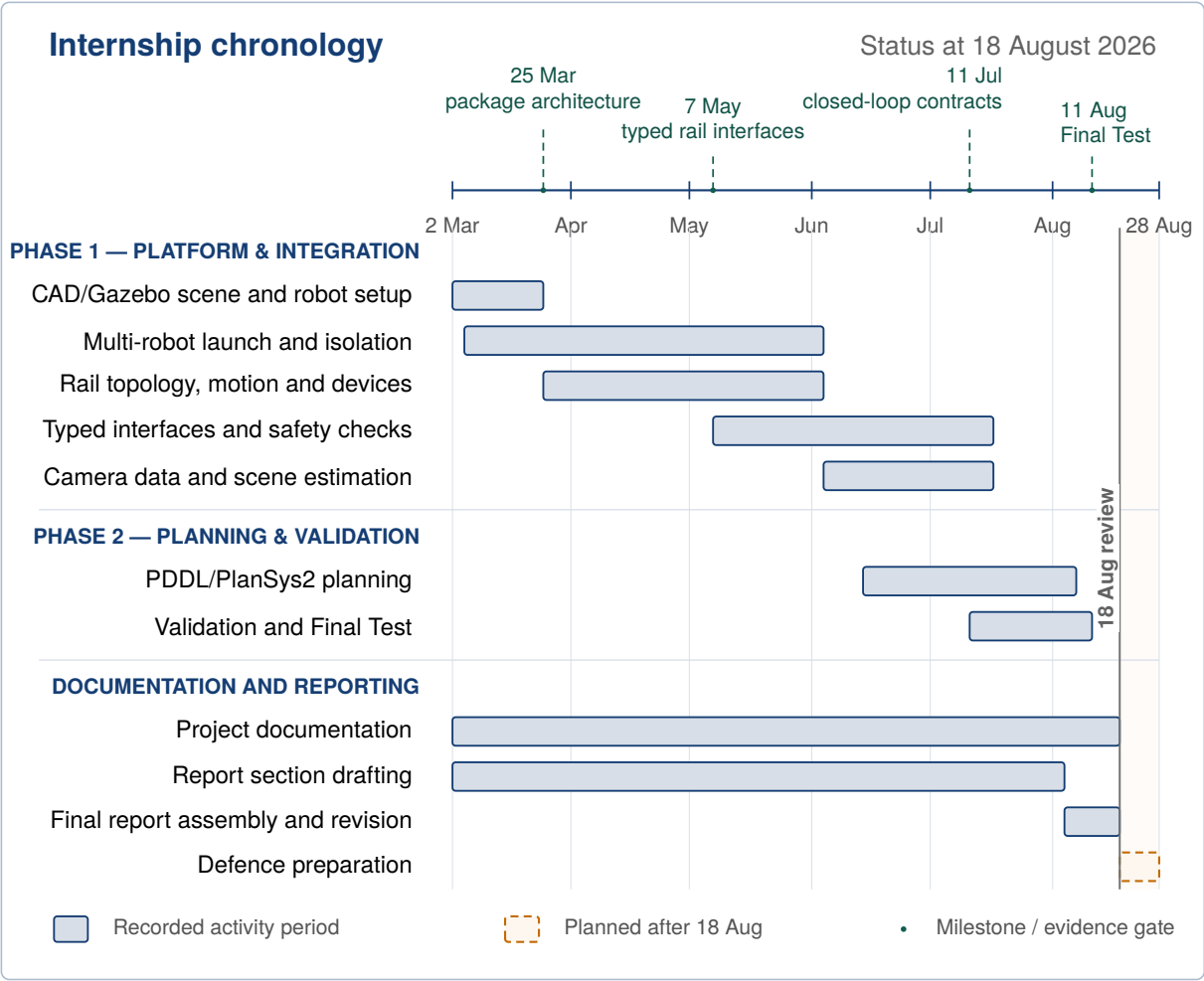


Figure 3.1: Technical chronology and remaining internship schedule. Project documentation continued throughout the internship. Report-section drafting ran from 2 March to 4 August, followed by final report assembly and revision through 18 August 2026. The dashed interval is planned defence preparation.

Each stage depended on a working result from the preceding one. Major technical decisions were reviewed weekly with the supervisors, then recorded in configuration and checked through tests and integration runs.

3.3 Requirements and review process

Requirements were recorded in operator options and guides, executable interface rules and tests, and hash-identified dataset/model manifests. They covered isolated robot communication, stable shuttle identity, sparse device updates, fresh scene state, deterministic plan-to-command translation and observation of every executed effect. Each property was checked at its implementation boundary; the safeguards are detailed in chapters 7 to 9, the artefacts in appendix A, and the objective assessment in table 11.5.

3.4 Personal responsibility, collaboration and resources

Within the shared repository, I designed the retained third-floor geometry in Autodesk Inventor and the furniture and static contents of both furnished rooms in the full-floor model. I exported and integrated them into Gazebo. For Room 315, I created the dedicated room model, including its doors and interior window. I then assembled the scene from the project-supplied rail-cell parts and the attributed robot models.

My software work covered selectable robot launch, calibrated rail motion, device behaviour and typed command and state interfaces. I also implemented the task checks, PDDL/PlanSys2 integration and one-action-at-a-time supervision. I created the paired-camera dataset and training process, tests, inventories, operating guides and handover records.

Weekly reviews with Diane Bury and Olivier Stasse were the principal coordination mechanism. I presented completed work, observed problems and technical alternatives. After discussing their feedback, I implemented and validated the agreed solution. Their institutional roles are shown in figure 2.2.

The technical chapters distinguish the first available host-supplied revision, project and supervisor input, reused foundations, my contribution and the validation I performed. The baseline does not imply an earlier repository state that was not available for inspection.

No purchasing budget was assigned to me. Resource management instead concerned the fixed schedule, access to the shared platform, graphics processing unit (GPU) time for model training, simulation time, dataset and saved-model storage, and the testing workload. I kept compute-intensive camera-processing and language-model components optional and prioritised reusable configuration and regression tests that detect whether changes break existing behaviour.

3.5 Autonomy and reviewed engineering decisions

I divided the integration objective into reversible, testable increments and reviewed decisions affecting shared interfaces, evidence or safeguards before finalising them. Table 3.2 records four situations in which supervisor guidance and my implementation responsibilities were distinct.

Table 3.2: Professional interaction, engineering decisions and project continuity.

Situation	Interaction and engineering decision	Outcome
Phase 1 completed earlier than planned	The supervisors proposed Phase 2 and we discussed its scope in the weekly reviews. I then translated this direction into the implemented neuro-symbolic language, vision and planning architecture.	The internship expanded from simulation modelling to the complete Phase 2 research workflow.
Incomplete physical rail parameters	With mass, contact and friction data unavailable, I discussed fidelity with the supervisors and proposed a calibrated kinematic model instead of unsupported dynamics. I implemented it and tested its geometry, routing, feedback and arrival.	Routing and planning experiments became repeatable without unsupported mechanical claims.
V3 cross-rail interference	I presented the observed limitation during the reviews, redesigned the higher-level model into rail-isolated branches and validated V4.	Development results improved substantially and the unwanted late cross-camera interaction was removed.
Continuity after the internship	I organised the project in the shared repository, prepared installation instructions, a quick-start guide and a runbook, and demonstrated the principal workflows to the supervisors.	Future students and developers can reproduce, operate and maintain the environment without depending on my presence.

3.5.1 Choosing PDDL/PlanSys2 over direct vision–language–action control

Phase 2 first explored a limited VLA-style event-selection prototype. It used planner-generated labels rather than human action demonstrations and produced no valid closed-loop result; one run was excluded after detecting target leakage. Room 315 is a finite domain, and the retained corpus labels scene state rather than actions. I therefore chose inspectable PDDL/PlanSys2 plans and kept learning at the checked language and visual-state boundary. The evidence and trade-off are detailed once in section 4.5.

3.6 Competencies and professional learning

The reviewed situations above document how decisions were reached; the table below summarises the engineering areas applied across the complete project.

Table 3.3: Engineering competencies and concrete project examples.

Competency	Project example
Systems analysis	Separation of geometry, scene estimation, planning, safety checks and the single component allowed to issue commands
Robotics integration	Robot models and controllers in ROS 2/Gazebo, selective start-up, isolated robot communication and device feedback
Algorithm design	Distance-calibrated rail graph, route planning, section reservations, safe spacing and result checks
Machine-learning methods	Local language-model integration, paired-image datasets split to prevent training/evaluation leakage and separate predictions for each scene property
Software quality	Explicit message contracts, focused regression tests, configuration inventories and reproducible result records
Project management and communication	Incremental milestones, supervisory reviews, diagrams, operating guides, result tables and explicit scope

Incomplete source data, a fixed schedule and limited computing resources shaped the modelling choices. Version control and testing preserved the evidence supporting those choices, as reported in Chapter 11.

Chapter 4

Technical Background and Design Choices

Simulation, transport, planning and visual estimation required explicit responsibilities so that the complete system remained inspectable and testable. The necessary ROS 2, Gazebo, rail and planning concepts lead to a comparison of learning and control architectures and the rationale for the final layered design.

4.1 ROS 2 as the integration layer

ROS 2 (Robot Operating System 2) is an open-source robotics software framework that connects processes called *nodes* through typed communication. Topics carry streams, services implement request–response operations and actions support longer tasks with feedback [23]. Fixed message schemas and namespace prefixes prevent ambiguous data and name collisions between robots; chapters 6 and 8 apply these mechanisms. Quality of Service (QoS) configures delivery properties [26]; because cameras, state and commands have different timing needs, the implementation also checks that safety-relevant reports are fresh.

4.2 Gazebo Harmonic and description formats

Gazebo Harmonic is a simulator release [21]. I used it with ROS 2 Jazzy on Ubuntu 24.04 for this project. The Simulation Description Format (SDF) describes worlds, models, physics, plugins and sensors [27]. The Unified Robot Description Format (URDF) describes robot link–joint trees, from which `robot_state_publisher` publishes coordinate transforms [34]. `ros_gz_bridge` translates selected messages between ROS 2 and Gazebo Transport [22]. In this project, I used these tools for interface checks, teaching, repeatable rail and safety scenarios, and labelled camera-data generation.

These purposes require different levels of fidelity. Detailed visual meshes support cameras, and simplified collision geometry is used where contact is modelled. The industrial arms deliberately omit collision elements, so they are debug models rather than contact-validation models. The rail study likewise uses repeatable kinematic poses instead of unmeasured wheel–rail forces (chapters 5 and 7).

ISO 23247-1 provides an overview, terms, definitions and requirements for a manufacturing digital-twin framework [15]. In this project, I used that standard as a general framing reference and implemented a versioned simulated environment with defined information flows, assumptions and boundaries; visual similarity alone was insufficient.

4.3 Coordinate and rail representations

A rail path is represented by piecewise geometric segments. Each segment has an arc-length coordinate s , a physical length L , and a normalised coordinate

$$\rho = \frac{s}{L}, \quad 0 \leq \rho \leq 1. \quad (4.1)$$

The controller evaluates pose and heading from s ; cubic Hermite interpolation joins ordered computer-aided-design (CAD) samples stored as comma-separated values (CSV) with continuous tangents. A separate graph defines legal connections for each switch state: geometry determines *where the rail is*, while topology determines *which route is open* (figure 4.1).

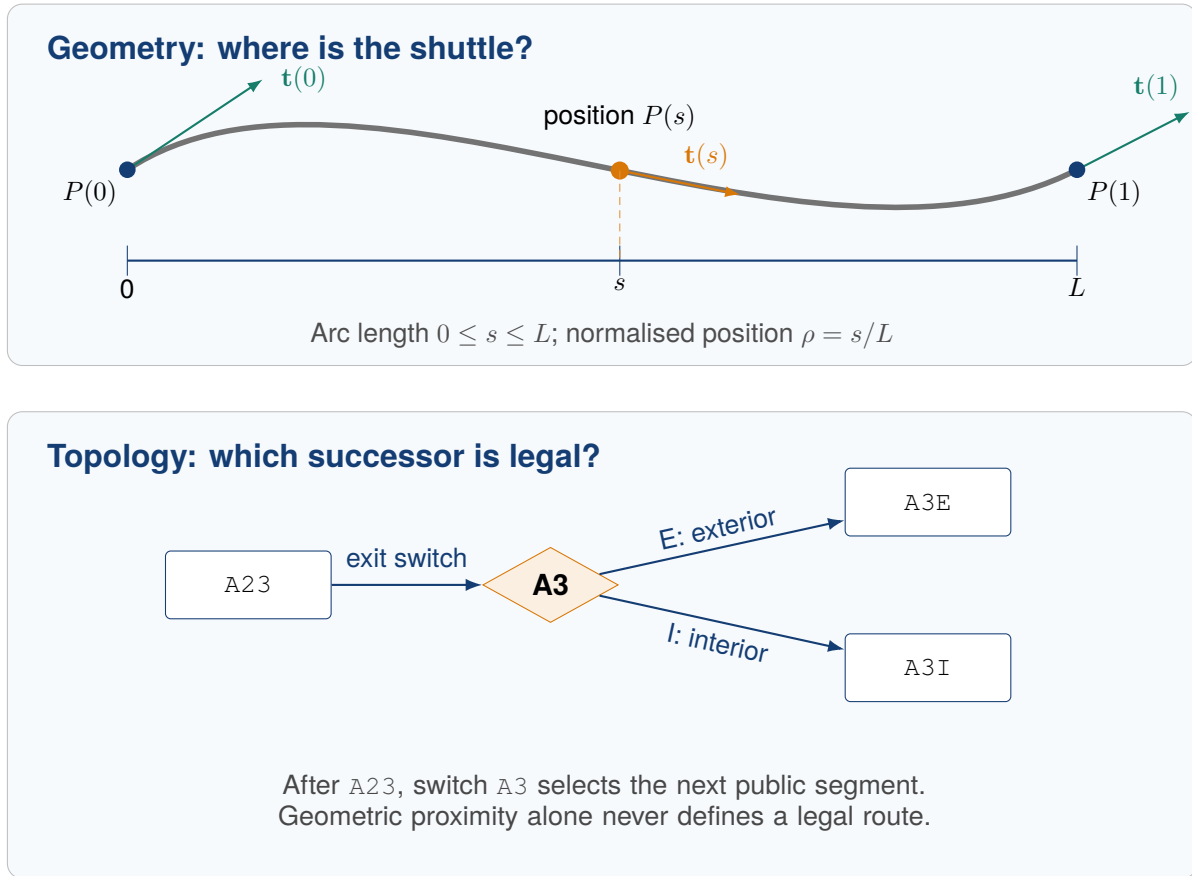


Figure 4.1: Complementary rail representations. Hermite geometry supplies position and tangent for pose construction; topology and switch state select the valid successor exposed through the public interface.

The ratio supports relative placement, while s preserves metric distance. Implementation and arrival semantics appear in Chapters 7–10.

4.4 Symbolic task planning

The Planning Domain Definition Language (PDDL) describes objects, predicates, actions, preconditions and effects [19]. PlanSys2 integrates this model with ROS 2 [18], while POPF searches for a sequence satisfying the stated goal [5].

A typical Room 315 planning task is to move a selected shuttle from its current slot to a requested target slot. Symbolic planning suits this task because the decisions are discrete and relational. The state records which shuttle occupies the source slot, whether the target and route are free, which branch is selected, whether devices are ready and whether the selected shuttle reached the target. A blocker is another shuttle occupying the requested route; when necessary, the plan first stages it temporarily on a designated interior holding segment.

PlanSys2 supplies named actions that prepare devices and request motion. The controller handles continuous travel; project-specific translation, supervision and effect checks protect each step (chapter 10).

4.5 Architectural alternatives and final design

Vision–language–action architectures assign learning different responsibilities. RT-2 [45] and TinyVLA [43] map observations and language to actions; SayCan ranks predefined skills [12]; OWL-TAMP supplies vision-language-model-generated constraints to a task-and-motion planner [16]. One recent comparison found neuro-symbolic methods outperforming VLAs on its structured

long-horizon manipulation tasks [8]. I therefore treat that result as task-specific, not as a universal ranking. Table 4.1 relates these patterns to Room 315.

Table 4.1: Representative architectures compared by the roles assigned to learning, planning and command execution.

Approach	Learned output	Planning role	Execution path	Decision for Room 315
End-to-end VLA policy	Robot-action tokens or trajectories	Action selection inside the learned policy; no separate symbolic planner	Decoded action passed to the robot control stack	Explored but not selected; the rationale follows the table.
Language/vision model with skills or a planner	Ranked skills, subgoals or planning constraints	Skill selector or task-and-motion planner	Selected skill or downstream controller	Related pattern; open-world constraint generation was unnecessary here.
Neuro-symbolic planning with learned control	Symbolic abstractions and learned low-level control	A PDDL planner determines the high-level sequence	Learned controller executes each planned PDDL action	Closest architectural family; adapted because Room 315 learns task and scene inputs but retains an explicit controller.
Implemented Room 315 system	Validated task fields and paired-camera state; neither is a plan or action	PlanSys2/POPF normally; the executive may prepend one route-normalisation step	Software execution supervisor publishes a typed command; the rail controller applies it	Selected project architecture; the rationale follows the table.

Decision for Room 315. The first exploratory architecture considered for Phase 2 used VLA-style direct action: task language, camera frames and structured observable state would select a compact symbolic event for software-supervisor validation and translation, not a motor command. An offline model was trained on planner-generated event labels, yet those traces imitated the existing planning workflow rather than providing independent human demonstrations or continuous robot trajectories. One learned-action run was also excluded because target-correlated fields had leaked into its input, and the direct-action path established no valid closed-loop result.

Room 315 instead has a finite, known decision domain: shuttle identities, slots, directed routes, devices and admissible actions can be enumerated. The retained V4 corpus labels scene state, the uncertain input needed by the planner. I therefore encoded the known rules as PDDL preconditions and effects and used PlanSys2/POPF to produce an inspectable, testable and traceable sequence. Learning remained useful for English understanding and visual-state estimation; neither model selects or executes an action.

Creating a leakage-free sequential corpus and independently qualifying a learned-action policy would have added substantial data, integration and verification work within the remaining fixed schedule. The layered design was therefore the lower-risk fit for the available knowledge, final data and verification method. This project-specific choice trades open-world flexibility for manual PDDL engineering; it does not show that symbolic planning is always superior.

Learned models interpret the task and scene. PlanSys2/POPF produces the symbolic sequence, while deterministic software authorises commands and checks their effects.

Chapter 5

Building the MFJA Digital Twin

I first built the third-floor CAD and Gazebo scenes, then integrated the robot assets and reusable launch profiles. The resulting shared simulation underpins the later transport, camera and planning work.

Contribution boundary

Start-of-internship baseline: The starting repository contained a minimal ROS/Gazebo scaffold: ground and sun, one basic world and launch file, and preliminary floor STL exports. It contained no completed third-floor model, furnished interiors, dedicated Room 315 world, robot registry or rail runtime.

Project and supervisor input: The supervisors provided platform knowledge and technical review; project resources provided equipment access.

Reused foundation: I reused ROS 2, Gazebo, project-supplied conveyor and rail CAD parts, and the robot descriptions attributed below; their source geometry is not my work.

My contribution: I designed the retained third floor in Autodesk Inventor, exported it for Gazebo, and designed and placed both rooms' furniture and static contents. I built the Room 315 model, integrated its robot workstations and rail cells, composed the operational worlds and launches, and organised the repository into four packages.

Validation performed by me: I checked dimensions, scale, orientation, coordinates and collision representations, added static world and robot checks, and ran the retained six-robot cold-start smoke test reported in chapter 11.

5.1 CAD modelling and Gazebo integration strategy

I iteratively exported my Autodesk Inventor third-floor design to Gazebo. The earliest retained stage exposed a missing room, incorrect door dimensions, incomplete windows, an unsuitable ground plane and a mesh-orientation error. I revised the source CAD rather than hiding those discrepancies with unrelated Gazebo offsets, then added the furniture and static contents of both furnished rooms. I created the dedicated Room 315 shell and assembled its workstations, guards and supplied rail-cell parts. The attributed robot geometry was reused; my work was its adaptation, placement and integration.

The robot-body assets have package-level provenance. KUKA uses `kuka_kr6_support` [37]. Stäubli uses `staubli_tx2_60_support` and shared TX60 meshes [35, 36]. Both Yaskawa variants use `motoman_hc10_support` [38]. TIAGo uses `tiago_description` from the Tiago-Harmonic fork of PAL Robotics' `tiago_robot` [39]. I adapted these assets through local URDF/SDF wrappers, paths and simulator integration; I did not create their robot-body geometry.

Each asset required consistent units, origin, pose, visual material and an explicit collision-model decision. CAD origins are often inconvenient manufacturing datums, while Stereolithography (STL) files contain no unit metadata. Explicit scale and orientation checks therefore aligned the visual geometry and, where enabled, the collision geometry with the intended room coordinates. Integration proceeded in three stages:

1. load each asset alone and verify scale, orientation and reference frame;
2. decide whether collision geometry was required, then place the asset in common coordinates; and
3. integrate it in the full scene and add static path/SDF checks.

Static room and rail-cell assets retain collision geometry where needed. The four industrial-arm models omit it for deterministic debugging, so they cannot test contact or collision avoidance.

Engineering iteration. The first useful check was not visual similarity at one camera angle, but whether known room features kept the same scale, orientation and relative position when loaded separately and in the complete world. A failed check returned the work to the Autodesk Inventor export or to the explicit SDF transform, depending on whether the discrepancy belonged to the source geometry or its simulation placement. After integration, static tests checked referenced assets, the deliberate presence or absence of collision elements and initial robot joints. This process produced a traceable final scene. No independent metrology was available to quantify geometric error.

5.2 World decomposition

Three configurations cover the shared third floor, focused Room 315 experiments and isolated robot checks. Launch passes the selected world’s internal Gazebo name so that entity and pose services remain aligned with it. Their retained entry points and intended uses are summarised in table 5.1; the complete arguments and commands remain in the runbook.

Table 5.1: Principal world and launch profiles.

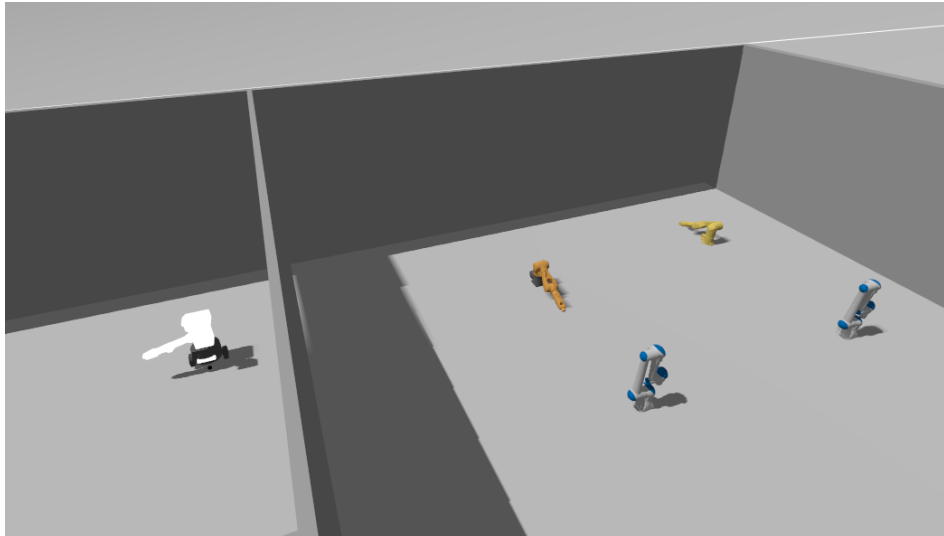
Profile	Entry point	Purpose
Full third floor	<code>full_floor.launch.py</code>	Shared spatial context, furnished rooms and selectable multi-robot integration
Room 315 only	<code>room_315_only.launch.py</code>	Faster rail, camera, planning and supervised-execution experiments
Isolated robot	<code>single_industrial_robot.launch.py</code>	Asset, namespace, bridge and command-interface checks without the complete floor

Figure 5.1 shows the reconstruction.

5.3 Repository refactoring

ROS 2 organises software into packages [25]. As the project grew, I split the flat layout into four of them: description assets, user-facing bring-up, typed rail interfaces and control/configuration. This separates reusable interfaces from their implementation and lets bring-up compose the system without owning control algorithms. `colcon` is the workspace build tool [4]; I used it to keep the workspace selectively buildable. figure 5.2 shows the declared internal package dependencies and appendix B lists the package contents.

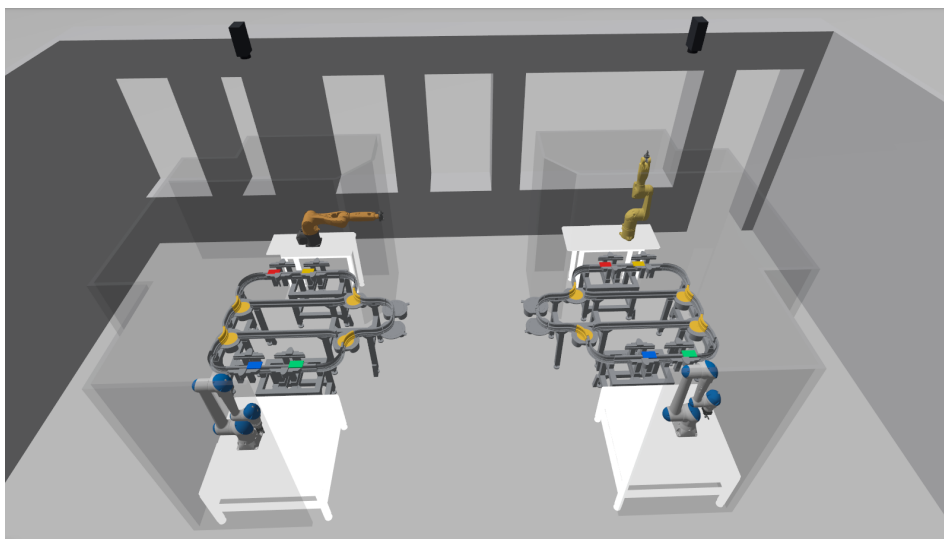
The initial flat package was adequate while it held only a world and a few assets, but adding robot wrappers, rail control, custom messages, planning, perception and tests made ownership ambiguous. In particular, a launch package that also defined message types or controller code would force unrelated users to depend on the complete research stack. I therefore kept models and worlds in the description package, made the public rail contracts independently buildable, placed runtime implementations and tests in control/configuration, and reserved bring-up for composition. After the move, I checked resource paths, package dependencies and the retained launch profiles from a clean build. The refactor did not change runtime message direction; it made build-time ownership and handover boundaries explicit.



(a) Earliest retained reconstruction stage.



(b) Third-floor CAD model I designed, rendered in Gazebo.



(c) Room 315 with the integrated rail cells and robot models.

Figure 5.1: My CAD-to-Gazebo work progressed from the earliest retained intermediate stage to the complete Autodesk Inventor floor geometry and the detailed Room 315 integration.

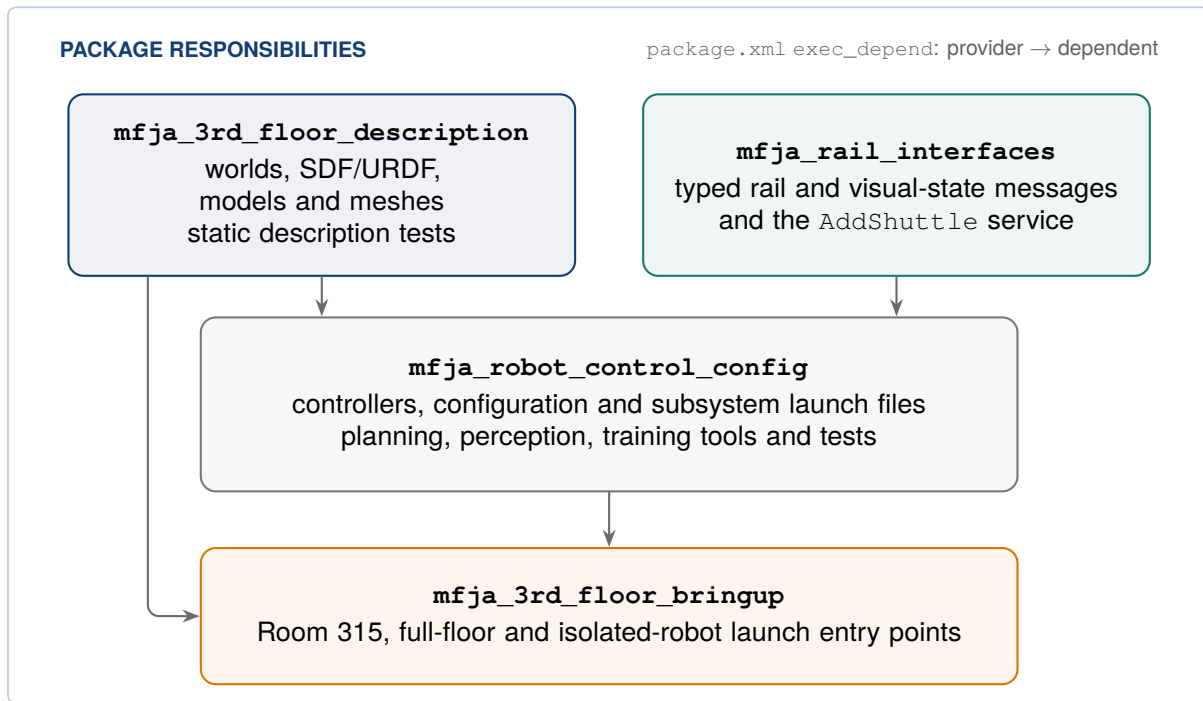


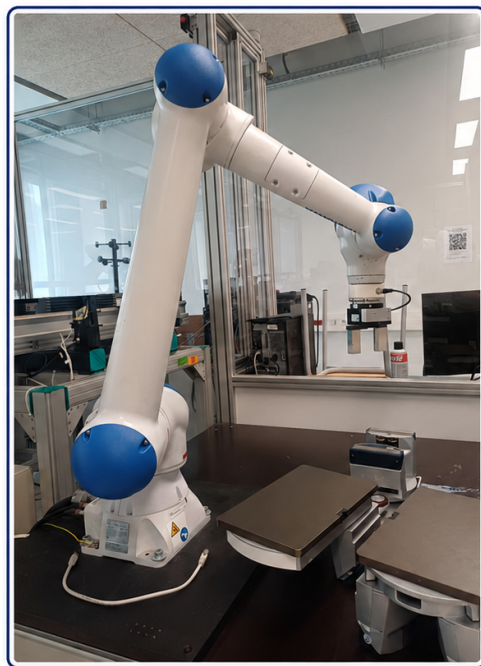
Figure 5.2: Responsibilities and internal dependencies declared in the four ROS 2 package manifests. Each arrow points from the package supplying a dependency to the package that declares it; the arrows do not represent ROS message flow.

5.4 Model integration

For each fixed-base arm, integration preserved the vendor link-joint tree and aligned its visual geometry with the controller joint list. TIAGo required a different composition because its mobile base, torso, arm and head expose several control groups rather than one fixed-base arm. These differences motivated the common access layer in chapter 6 without pretending that all robots share one command surface.

5.4.1 Qualitative physical-to-simulation comparison

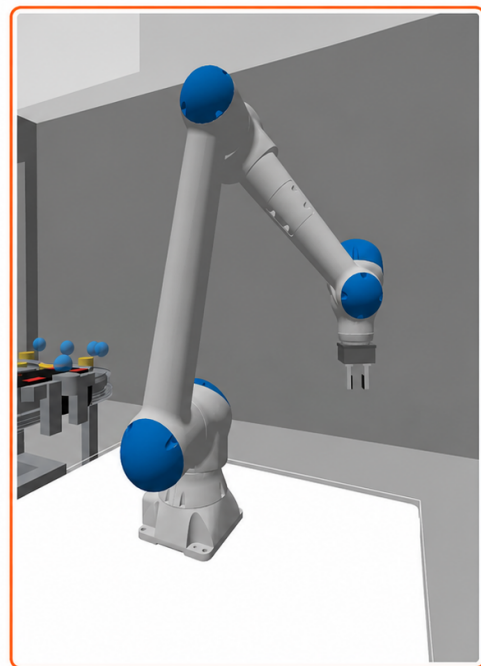
Figures 5.3 to 5.5 compare each physical platform with its integrated Gazebo model. They check recognisable morphology, joints and tooling. The comparisons are qualitative because the view-points differ.



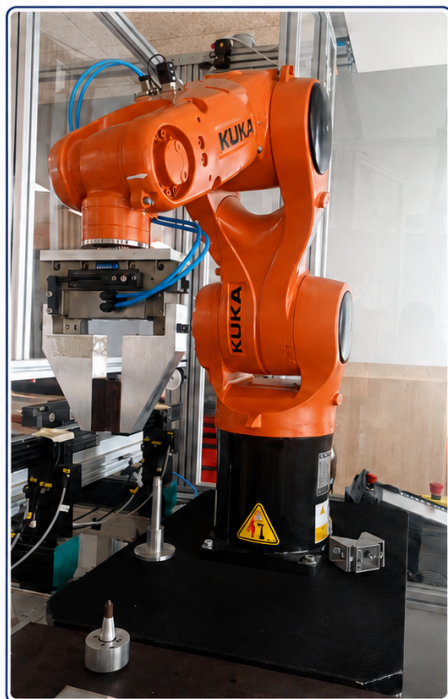
Real robot setup
Yaskawa HC10



Reality to
simulation



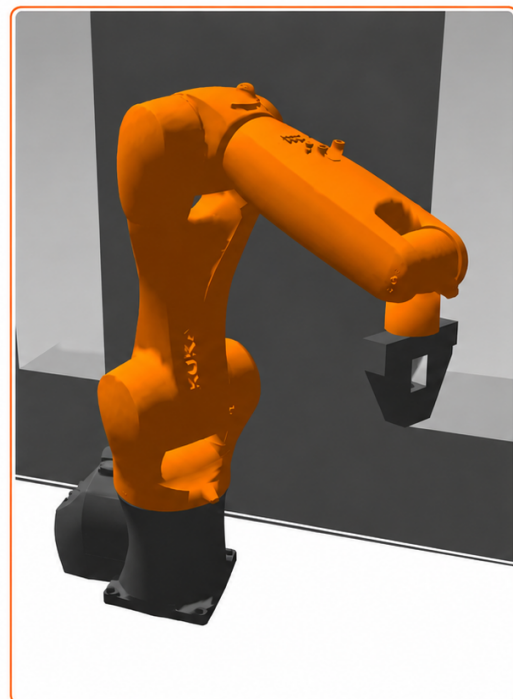
Gazebo simulation model
Yaskawa HC10



Real robot setup
KUKA KR6



Reality to
simulation



Gazebo simulation model
KUKA KR6

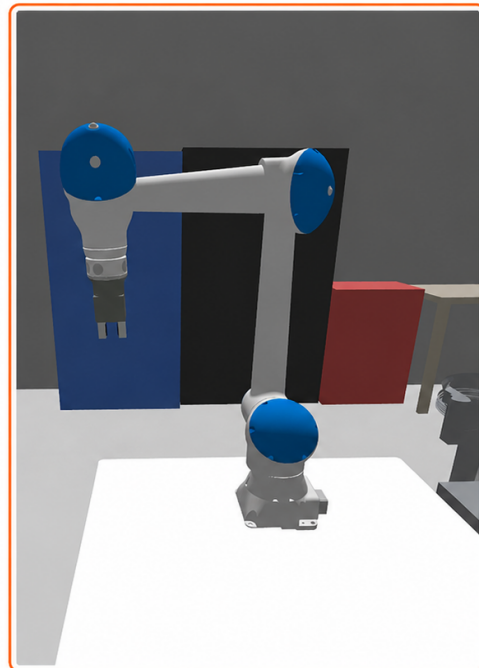
Figure 5.3: Qualitative physical-to-Gazebo comparison of the industrial workstations beside the left rail: Yaskawa HC10 (above) and KUKA KR6 R900 sixx (below). Each panel shows the physical installation on the left and the integrated Gazebo model on the right.



Real robot setup
Yaskawa HC10DT



Reality to
simulation



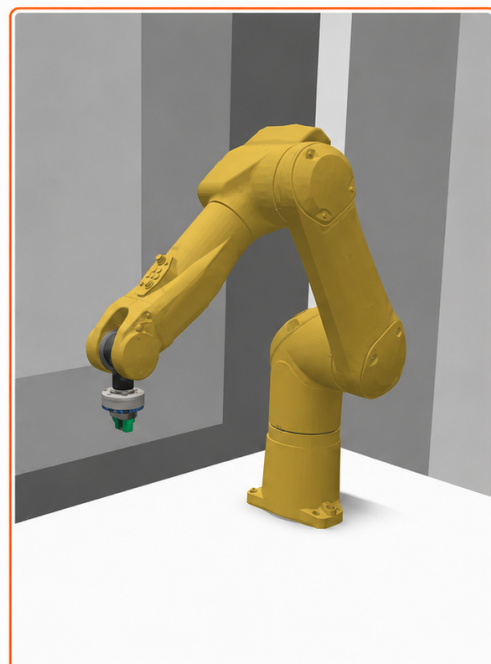
Gazebo simulation model
Yaskawa HC10DT



Real robot setup
Stäubli TX2-60L



Reality to
simulation



Gazebo simulation model
Stäubli TX2-60L

Figure 5.4: Qualitative physical-to-Gazebo comparison of the industrial workstations beside the right rail: Yaskawa HC10DT (above) and Stäubli TX2-60L (below). Each panel shows the physical installation on the left and the integrated Gazebo model on the right.

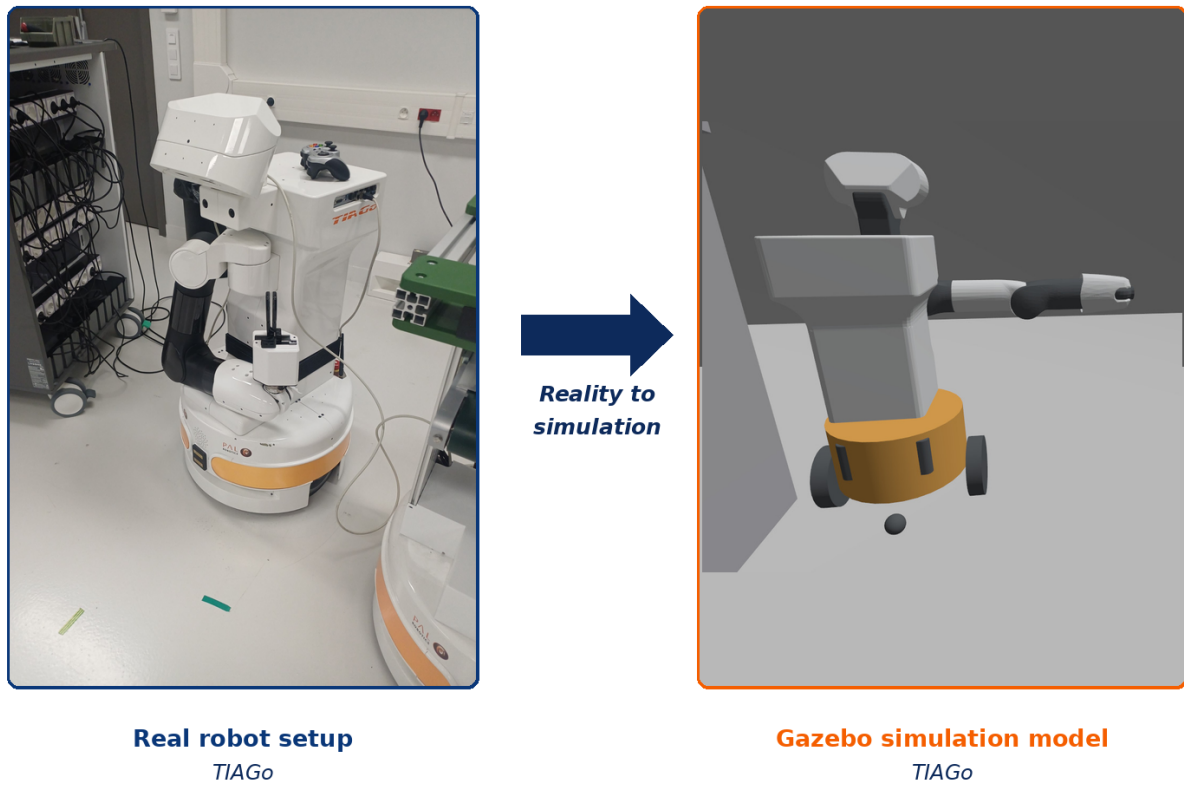


Figure 5.5: Qualitative physical-to-Gazebo comparison of TIAGo: the physical platform is shown on the left and the model integrated in the shared Gazebo environment on the right.

5.4.2 Room and device assets

Room 315 also has separate cell, rail, device, shuttle, payload and camera assets; capture can hide inspection markers, and payload is *loaded* or *empty*.

5.5 Launch profiles and reproducibility

ROS 2 provides a configurable package launch system [24]. I used it to reuse one Room 315 argument set for full-floor, Room-only and isolated-robot modes. Operators select robots, shuttles, cameras, markers, initial slots and graphical or headless operation; research components require explicit activation. These profiles isolate an integration boundary or scale without changing public namespaces; their entry points, runbook and status are indexed in appendix B and chapter 11.

The Ubuntu 24.04, ROS 2 Jazzy and Gazebo Harmonic workspace installs the recorded runtime. The full-floor smoke covers startup and interface availability; appendix A gives the build record.

5.6 Conclusion

The retained third-floor and Room 315 scenes now load through reproducible profiles with integrated robot assets. Static checks and the six-robot cold-start confirmed loading, isolated names and fresh feedback. The main difficulty was aligning CAD units and origins while documenting each collision-model decision.

Chapter 6

Unified Multi-Robot Access and Control

Six different robot models had to run together without losing their model-specific command interfaces. I linked a shared registry and instance selector to isolated communication links and a common command tool.

Using the project-supplied inventory and the robot models attributed in chapter 5, I designed and tested a registry in YAML, a human-readable configuration format. I also implemented selector rules, isolated namespaces, bridge generation, the launcher and the command helper; none of this shared access layer was present in the baseline.

6.1 What “unified” means

Here, *unified* means that one YAML registry declares selectable instances, one launch path starts them and their bridges, one separate helper validates a named robot and joint vector, and instance-derived namespaces isolate commands and feedback.

A joint trajectory is a timed sequence of joint targets. Fixed-base arms use these trajectories, whereas TIAGo also exposes base velocity and a linear torso joint. The helper validates the selected robot, vector and units without hiding these model-specific interfaces.

6.2 Multi-robot operation in one Gazebo process

Each selected YAML entry defines an instance name, model, pose and enabled state. Launch rejects duplicate names and derives its namespace and bridge from that identity; the helper supplies the corresponding joint order, topic and units. Selection, spawning, commands and feedback therefore share one identity chain. The retained smoke covers all six robots, while automated tests cover the remaining selector forms (chapter 11).

6.3 Configuration-driven robot registry

The full floor and Room 315 use the same loader with different YAML files. Selectors accept a full name, alias, numeric order, set, `all` or `none`; the retained instances and their principal command surfaces are listed in table 6.1. A separate validated Python table records aliases, joint order, topics and unit semantics. This remaining duplication is test-covered, but the two configuration sources are not generated from one definition.

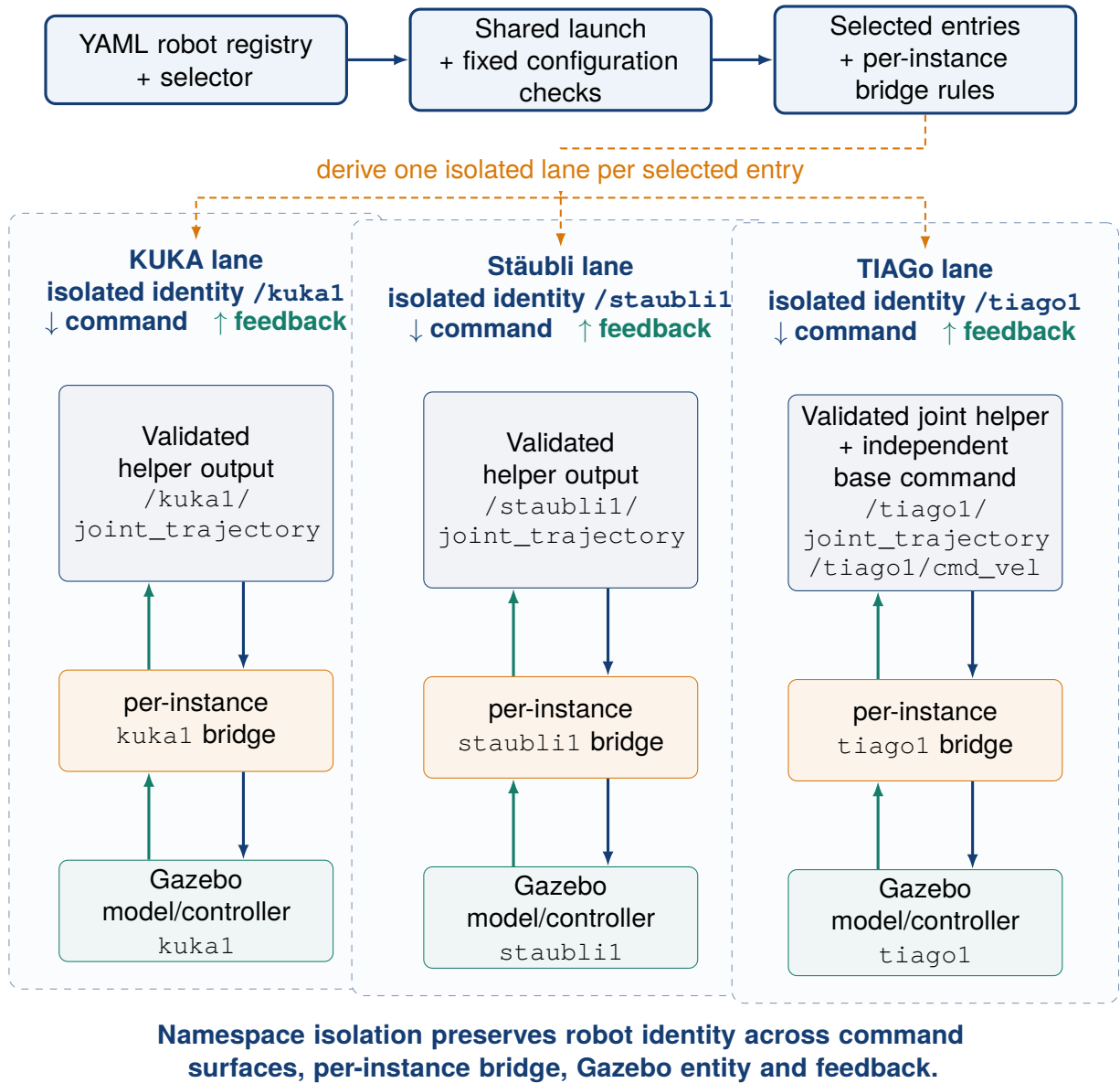
6.4 Dynamic bridge generation

`ros_gz_bridge` translates between Gazebo Transport and ROS 2 messages [22]. In my implementation, launch derives one bridge lane per selected YAML identity. Commands enter Gazebo and required state, odometry and transforms return on that lane, preserving one-to-one addressing (figure 6.1).

Deriving the bridge from the selected registry entry is important: a renamed instance changes its spawned entity, ROS namespace and bridge topics together. A hand-written bridge list could leave one of those three names stale and route a valid command to the wrong model. The loader therefore rejects duplicate instances before spawn, and tests compare the selected identity with the topics created for it.

Table 6.1: Robot selectors and principal command surfaces.

Selector	Instance	Type	Primary command
kuka	kuka1	KUKA KR6	/kuka1/ joint_trajectory
staubli	staubli1	Stäubli TX2	/staubli1/ joint_trajectory
hc10	yaskawa_hc10_1	Yaskawa HC10	/yaskawa_hc10_1/ joint_trajectory
hc10dt	yaskawa_hc10dt_1	Yaskawa HC10DT	/yaskawa_hc10dt_1/ joint_trajectory
tiago	tiago1	Mobile manipulator	Joint trajectory and /tiago1/ cmd_vel
tiago_base	tiago_base1	Mobile base with torso	Joint trajectory and /tiago_base1/ cmd_vel

**Figure 6.1:** Selected YAML entries generate isolated per-instance bridges and helper addressing; there is no shared global joint-command topic.

6.5 Joint-command helper

The helper validates robot identity, position count, joint mapping and units before namespaced publication; it does not plan trajectories, collision-free motion or cell synchronisation. Typed boundaries allow another provider to preserve identity, units, timestamps and feedback. Usage and provenance are in appendix A.

Publishing a raw `JointTrajectory` remains possible, but the helper removes two common operator ambiguities: the order of the numeric positions and whether an angular value is expressed in degrees or radians. Linear joints remain in metres. A rejected vector produces no publication, while a valid vector is sent only to the selected instance namespace.

Table 6.2: Checks applied at the shared robot-access boundary.

Boundary	Rejected condition	Observable oracle
Selection and identity	Unknown or duplicate selector; inconsistent instance identity	Launch stops before spawn, so no ambiguous namespace or bridge is created
Command vector	Unknown robot, wrong value count or unsupported unit	The helper publishes no trajectory
Configuration parity	Alias, joint order or topic differs between the registry and helper table	The parity test fails before the configuration is used for a command
Feedback isolation	Missing fresh state or an identity/topic mismatch	The cold-start check fails unless every selected instance has distinct feedback

Automated selector, parity and helper tests cover the first three boundaries. The retained six-instance cold start verifies distinct, fresh feedback for the fourth.

Portability boundary. The public identity, command and feedback contracts are independent of the Gazebo mesh and could be preserved by a future hardware provider. Implementing that provider was outside this work.

6.6 Conclusion

The resulting access layer starts six isolated robot instances without hiding their command differences. The 42-check suite and cold-start smoke confirmed selector behaviour, distinct names and fresh feedback (chapter 11). The main integration difficulty was preserving one identity chain across different joint counts, units and mobile-base interfaces.

Chapter 7

Room 315 Kinematic Transport System

Because the available physical parameters were incomplete, I selected a kinematic rail model and converted the CAD centre-lines into a directed graph. The resulting controller applies the permitted directions, device behaviour, spacing and staging logic needed for repeatable routing tests.

I started from project-supplied rail CAD and the physical switch names A1–A4. I designed the segment vocabulary and topology, then implemented and tested the dual-rail controller, devices, separation, reservations and staging. No rail graph or controller was present in the baseline.

7.1 Modelling decision: dynamic or kinematic

I compared a dynamic rigid-body model driven by wheel–rail contact with a kinematic model that places each shuttle from its graph segment and travelled distance. Dynamic simulation would suit traction, slip or wear studies, but a credible eight-shuttle model required unavailable mass, friction, compliance, drive and switch data; apparent physical accuracy would therefore be unsupported. The project instead needed repeatable routing, sensor crossings, arrivals, camera scenes and supervised execution. I selected kinematic motion because it isolates geometry, topology and authorisation while producing those observations consistently. It does not claim wheel–rail dynamics.

7.2 Kinematic transport backend

The backend advances arc length on the calibrated graph and publishes poses through Gazebo’s `set_pose` service. Gravity and wheel contact do not drive the shuttle; controller state is authoritative, so rendering lag does not alter logical progression.

Before detailing the graph, figure 7.1 relates each physical transport cell to its integrated Gazebo counterpart. The views provide an orientation aid and a qualitative assembly check; the following sections give the calibrated geometry, topology and behavioural evidence.

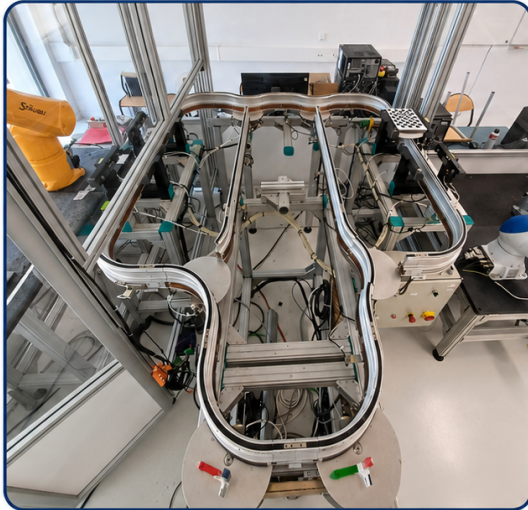
7.3 From SolidWorks points to a continuous rail graph

SolidWorks is a three-dimensional CAD application [6]. I used it to inspect the project-supplied rail model and export its centre-line points. Both rail configurations use the resulting 276 ordered points in 14 CSV curves. I split them at branch boundaries, preserved their direction, mapped each public segment to a curve and applied side-specific calibration before Gazebo publication. CSV files define *geometry*; YAML successor rules define *connectivity*.

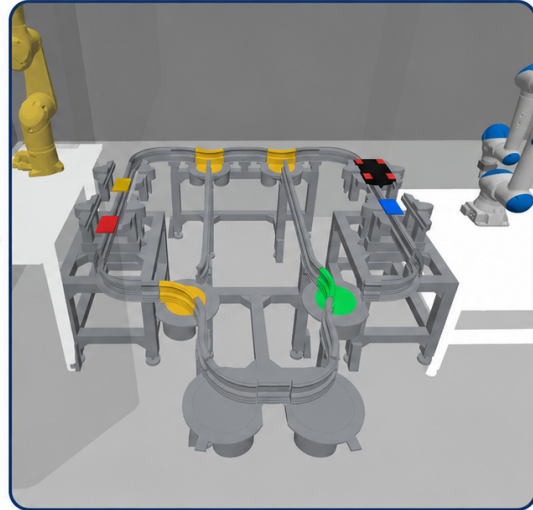
Each public segment has an arc length and the normalised coordinate $\rho = s/L$ from equation (4.1); the public interface names this value `s_ratio`. Cubic Hermite interpolation was retained because it joins samples with continuous heading; linear interpolation produced yaw jumps, while an unconstrained global spline could leave the rail. Tests compared the runtime evaluator with the source polyline and checked route and device mappings. Figure 7.2 shows the configured reconstruction; the data schema, transform and evaluator details are in appendix B.

REALITY TO GAZEBO SIMULATION

RIGHT RAIL

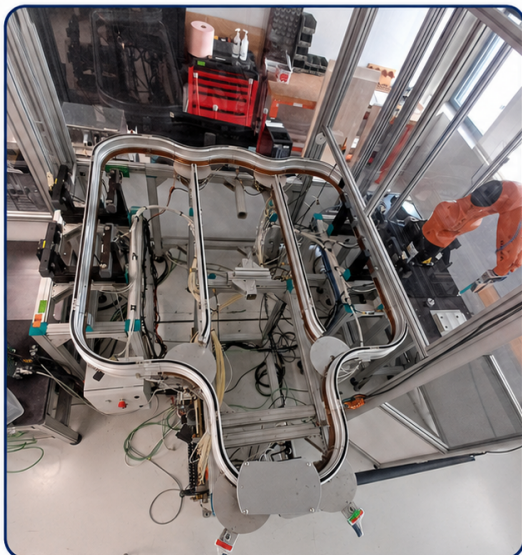


Real System

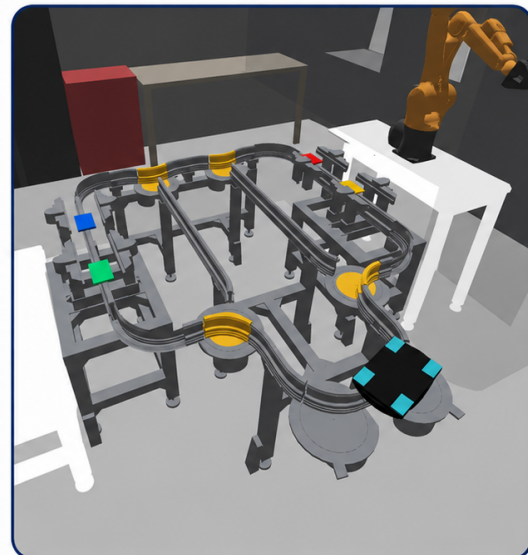


Gazebo Simulation

LEFT RAIL



Real System



Gazebo Simulation

Figure 7.1: Physical-to-Gazebo comparison of the two Room 315 transport cells: the right rail is shown above and the left rail below. The differing viewpoints make this a qualitative check of the loop and branch layouts.

Right-rail geometry: 276 CSV samples and runtime reconstruction

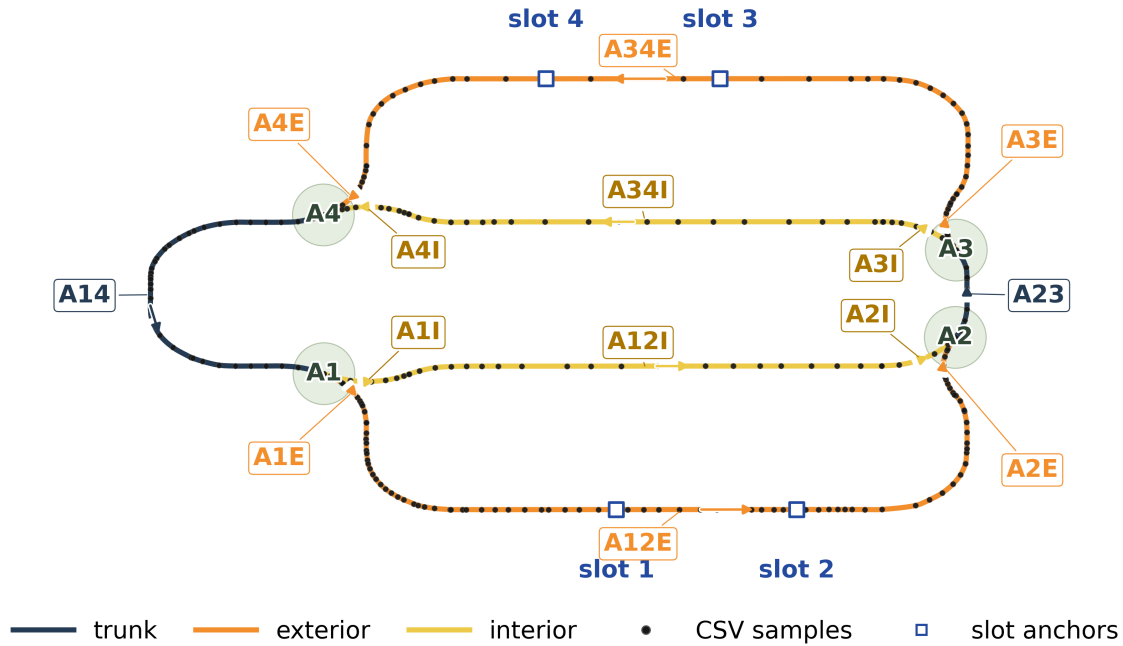


Figure 7.2: Room 315 centre-line from 276 ordered CAD/CSV samples: black dots are source points and coloured curves are dense Hermite outputs. The drawing shows the right-rail source/CAD orientation; the left rail reuses the samples with its own internal geometry identifiers and calibration. Affine calibration applies side-specific planar scale, rotation and translation plus height/yaw offsets before Gazebo publication. This checks the configured reconstruction; no independent metrology was available.

7.4 Topology, inherited switch names and segment vocabulary

The physical installation and project documentation already identified the four switch devices/zones as A1–A4; I retained these inherited identifiers. They name switches, not rail segments. Starting from them, I designed the 14-label public segment vocabulary shared by the controller, ROS 2 interfaces, visual labels and PDDL model.

In this convention, E means exterior and I means interior (the inner branch). A one-number form A_iE/A_iI denotes the short connector adjacent to switch A_i . A two-number form $A_{ij}E/A_{ij}I$ denotes the exterior/interior route between switch zones A_i and A_j ; for example, $A_{34}E$ is the exterior A3–A4 segment between the adjacent connectors A_{3E} and A_{4E} , while $A_{12}I$ is the interior A1–A2 segment. An unsuffixed two-number form is a shared single-track section, such as A_{23} or A_{14} .

When switch selections are consistent, the directed segment sequence is

$$A_{14} \rightarrow A_{1E/I} \rightarrow A_{12E/I} \rightarrow A_{2E/I} \rightarrow A_{23} \rightarrow A_{3E/I} \rightarrow A_{34E/I} \rightarrow A_{4E/I} \rightarrow A_{14}.$$

These labels and successor rules form the discrete topological language of my hybrid routing design; position within a segment remains continuous through arc length s . They are software interface states, not additional physical rails. Each shuttle moves only in the forward direction shown by the arrows; reverse travel is outside the controller and planning contracts.

Each switch has exactly two positions, EXTERIOR and INTERIOR. With four switches per rail, this gives $2^4 = 16$ assignments. The left/right network YAML files define the nodes, 14 segments, connections and switch-conditioned successors.

The controller and PDDL generator read this same public topology on both rails; left-side internal geometry names are normalised before publication. Rules, rather than point distance, select the

successors listed in table 7.1. In forward traversal A2 enters A23, then A3 selects its exterior/interior successor.

Table 7.1: Successor map for the public segment vocabulary shared by both rails.

Segment	Switch	State	Next
A23	A3	E	A3E
A23	A3	I	A3I
A3E	—	Any	A34E
A3I	—	Any	A34I
A34E	A4	E	A4E
A34E	A4	I	FALLING
A34I	A4	E	FALLING
A34I	A4	I	A4I
A4E	—	Any	A14
A4I	—	Any	A14

Segment	Switch	State	Next
A14	A1	E	A1E
A14	A1	I	A1I
A1E	—	Any	A12E
A1I	—	Any	A12I
A12E	A2	E	A2E
A12E	A2	I	FALLING
A12I	A2	E	FALLING
A12I	A2	I	A2I
A2E	—	Any	A23
A2I	—	Any	A23

Here, “Any” denotes an unconditional successor. An incompatible converging switch has no successor and enters FALLING: a latched no-valid-successor fault that stops at the segment end, not a physical fall.

7.5 Devices and their public identifiers

Switches A1–A4 select exterior or interior branches and report their state; stoppers hold or release approaching shuttles; binary sensors report occupancy and shuttle identity. DZI identifiers denote indexed slot detectors, while DA identifiers denote switch-approach or branch detectors. The target workflow prepares the stopper, requests motion and waits for the identity-bearing DZI; continuous controller pose is not a learned-model input. Exact names, states, protected distances and the device map are in appendix B; chapter 8 defines the supervisory vetoes.

7.6 Multi-shuttle identity and lifecycle

Each rail supports four stable identities: L1–L4 or R1–R4, mapped to Gazebo entities and never inferred from array order. The registry keeps configuration, world entities, controller reports, vision and planner objects aligned. It supports per-shuttle enable, disable, reset and removal, while emergency stop disables the fleet until an explicit resumption decision. Only the eight language-interface identities enter supervised execution; detailed service and mode values are in appendix B. Figure 7.3 shows the full fleet in both visual-pipeline cameras.

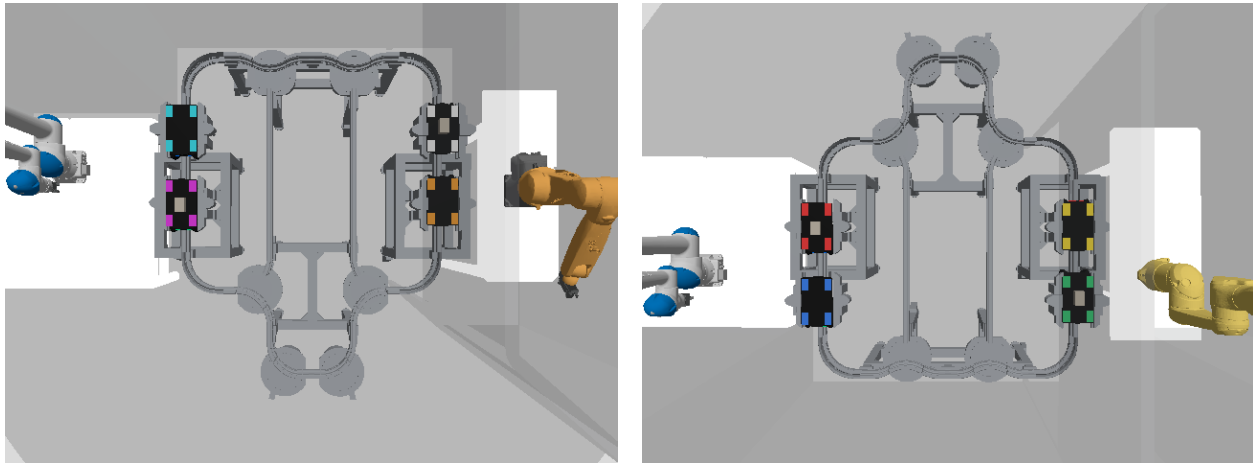
7.7 Motion, slots and occupancy

At every update step, an enabled shuttle advances by

$$s_{k+1} = s_k + v \Delta t, \quad (7.1)$$

where s_k is arc length at update k , v configured forward speed and Δt the interval. Route, target, stopper and headway constrain the update; crossing an end transfers residual distance to the topology-selected successor. Timing details are in appendix B.

Slots are calibrated exterior-segment `s_ratio` values. Arrival combines target, stopper, slot detector and OFF command effect: OFF reports DISABLED, while `reached_target_slot` records low-level setpoint completion, not task success. The planner tracks symbolic occupancy; the executive still verifies target identity.



(a) Left: L1-L4; L2 and L4 loaded.

(b) Right: R1-R4; R1 and R3 loaded.

Figure 7.3: Synchronous overhead Gazebo views of all eight shuttles. With device markers hidden, identity colours, presence and payload appearance remain visible.

7.8 Operator-facing observation and maintenance

The low-level API supports commissioning without bypassing supervised task execution. Its device-position tool projects a Gazebo point onto configured rail geometry, defaults to a non-writing preview and guards large-residual updates; changes take effect only after relaunch and feedback checking. Commands and maintenance procedures are listed in appendix B and the runbook.

7.9 Headway and collision constraints

The controller enforces a configurable default 0.33 m centre separation. This is a kinematic anti-overlap constraint, not a measured braking or machinery safety distance. Higher-level headway and reservations require clear route blocks ahead (chapter 8). A *blocker* is another shuttle occupying the route, destination or holding capacity; other objects are outside this model.

7.10 Interior staging and dense blocker cases

Interior branches A12I/A34I are capacity-limited holding areas; their calibrated length, margins and separation admit at most two shuttles. Successful staging records a clearance certificate linking identity, bounded motion, stop and post-stop observation. It supports route normalisation rather than visual localisation and is invalidated by later motion; chapter 10 covers recovery and appendix B the exact fields.

7.11 Conclusion

The result is repeatable motion on two calibrated rail graphs with stable shuttle identity and device feedback. All 96 transport checks passed across geometry, connections, devices, minimum spacing and route ownership (chapter 11), and the integrated campaign exercised blocked routes. The central difficulty was converting sampled CAD curves and one-way switch behaviour into one route map.

Chapter 8

Typed Interfaces, Contracts and Safety Supervision

I implemented typed contracts to separate task intent, commands, state reports and observed effects. The supervision layer uses those contracts to reject stale or conflicting state, protect shared routes and verify command effects.

Using standard ROS 2 interfaces and my Phase 1 rail simulator, I designed and tested the typed contracts, sparse-update rules, reservations, software command checks and effect verification; none of this supervision layer was present in the baseline.

8.1 Typed and versioned contracts

ROS 2 uses `.msg`, `.srv` and `.action` definitions for messages, request–response services and long-running goals [23]. I used these definition types for the rail interface. They keep four roles distinct: a *task* states the requested outcome, a *command* requests a device change, *state* reports what the controller observes, and a *result* records whether the expected effect was established. A PDDL effect is only the planner’s predicted change; the executive must confirm the corresponding simulator state from newer reports.

Switch and stopper commands are sparse named updates. For example, `switches: {A3: INTERIOR}` changes only A3; an omitted name retains its current value. After validation, the supervisor converts this checked envelope, represented in JavaScript Object Notation (JSON), to a typed `SwitchCommand`, `StopperCommand` or `ShuttleCommand`. Training labels and translator metadata are never accepted as live commands.

Operational records also use versioned schemas that constrain fields and types, record identifiers, timestamps and provenance, and restrict which component may produce each record. Incompatible versions and unauthorised producers are rejected before planning or command publication. The principal contract catalogue and producer matrix are retained in appendix B; figure 8.1 shows the two rules most important at runtime.

8.2 Accepted state and software supervision

Observed facts retain four explicit statuses: *known*, *unknown*, *stale* and *conflicting*. Only fresh, authorised and consistent known facts may populate an executable planning state; a missing reading is therefore not interpreted as false or as free space. The gateway combines the confirmed task with accepted visual, device and identity-bearing sensor reports while keeping learned estimates separate from exact simulator state and test-only providers.

The rule-based `room_315_vla_supervisor.py`—not a learned VLA policy—gives final software authorisation. It validates sparse JSON commands against rail state, then publishes typed commands consumed by the kinematic controller.

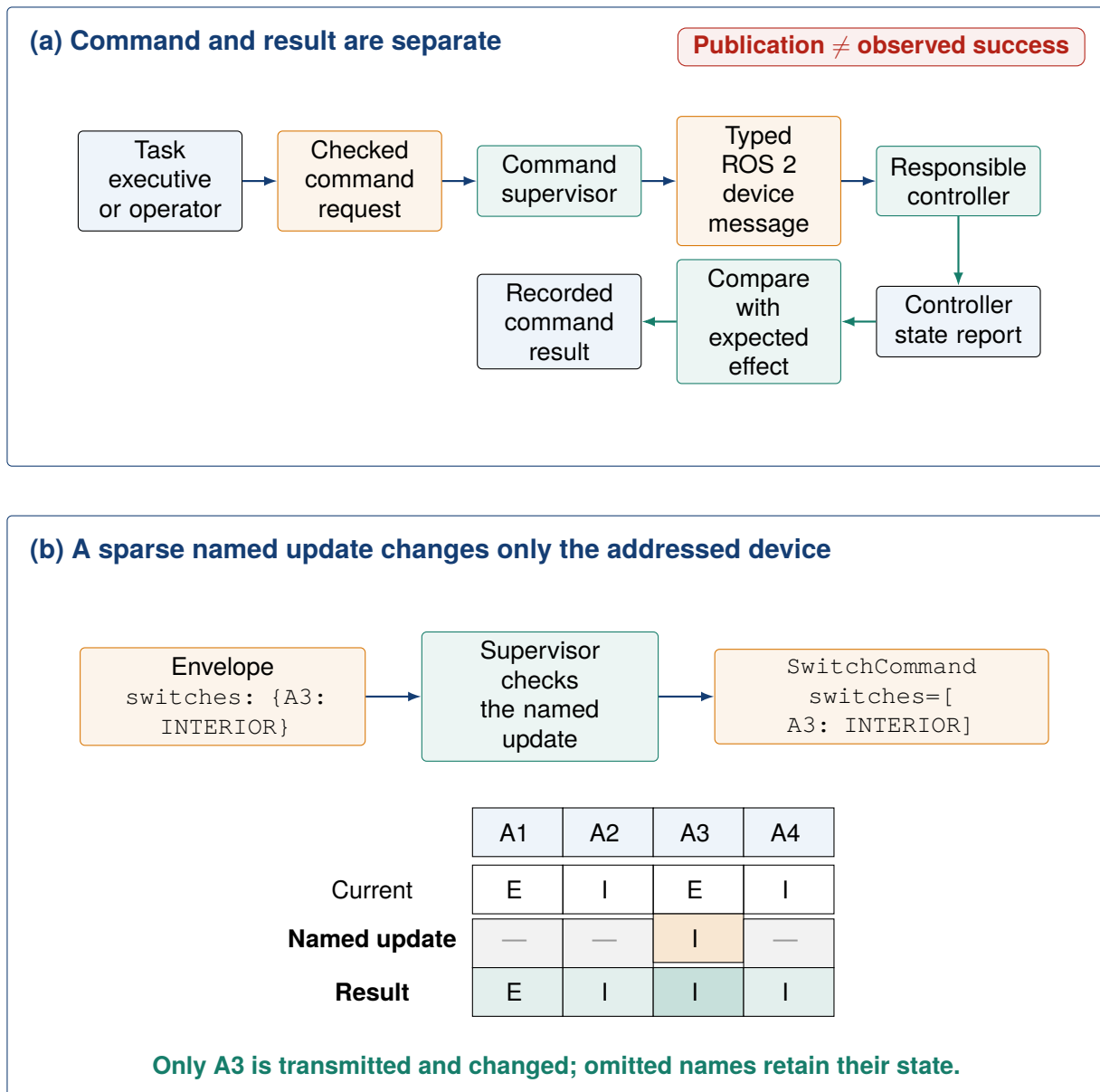


Figure 8.1: The supervisor checks sparse commands before typed publication; feedback establishes results and omitted devices retain state.

Before publication it checks:

- latched emergency-stop state and the immediate global `stop_all` operation; command structure, identity and rail-side consistency;
- fresh/non-conflicting state, occupied targets, reservation-block ownership, geometric separation and block headway;
- switch change within the 0.35 m protected distance; stopper and route-clearance state;
- post-command occupancy by another shuttle, sensor dropout, timeout, unknown localisation, impossible pose or `FALLING` mode;
- a configured recovery-retry limit (default two) and lexicographic reservation-conflict tie-breaking.

Diagnostics retain metrics, recent decisions and each authorisation/rejection reason; rejection leaves the device unchanged. Exceptions or shutdown during an active goal trigger gateway `stop_all`.

Reservations give one shuttle/task temporary ownership of a route block, while geometric headway independently prevents overlap. A crossed-ownership conflict uses deterministic identity priority; the other shuttle stops, observes and replans. Switch changes are additionally blocked when fresh controller coordinates place a shuttle within the protected rail distance. This handles the declared conflict pattern, not arbitrary wait-for cycles or temporal non-progress; the detailed resource and feedback rules are in appendix B.

8.3 Effect verification and emergency handling

The runtime defines the expected effects of every translated execution command. Examples include:

- `SET_SWITCHES/SET_STOPPERS`: requested values match controller status;
- `commanded slot motion`: two consecutive identity-bearing DZI readings, controller `reached_target_slot`, explicit `DISABLED` state and required confirmation frames agree;
- `STOP_NOW`: controller reports a fresh explicit `DISABLED` state;
- `observation-only inspection`: a newer visual observation is available after the request.

Results retain command, plan-step and observed-state identifiers; motion effects also record command-linked observations and require newer reports for arrival and retries. Switch/stopper matching permits *cached state*: the last accepted value may predate the command, unlike post-command motion checks.

Manual device-debug commands use normal structural/supervisor checks. The exception is emergency: it immediately invokes all-shuttle `stop_all` without planner state and latches rejection of motion until explicit clearing. Stoppers can thus be tested without a language model but remain subject to the same software checks.

8.4 Conclusion

The typed messages and rule-based supervisor now connect task intent to checked commands and observed effects; chapter 11 reports their verification. The main difficulty was combining partial device updates arriving at different times without treating missing values as false. Complex deadlocks, stalled tasks and machinery-safety certification are outside this supervisor's scope.

Chapter 9

Developing the AI Inputs: English Understanding and Visual-State Learning

Chapter guide. Learned models could assist with task and scene interpretation, but neither was allowed to issue commands. I therefore built a constrained English interface and a rail-isolated visual estimator that supply the planner with two checked records.

Contribution boundary

Start-of-internship baseline: The starting repository contained no Room 315 language pipeline, paired-camera corpus or trained visual-state estimator.

Project and supervisor input: The supervisors supplied platform knowledge and reviewed the methods and claims.

Reused foundation: I integrated Qwen2.5-1.5B-Instruct [32, 44] through `llama-cpp-python` v0.3.16 and `llama.cpp` [3, 10]. I also reused TorchVision’s ImageNet-pretrained ResNet-18 [7, 11, 31]; I did not develop these external models or tools.

My contribution: I developed the constrained language contract and dialogue pipeline, generated and partitioned the paired-camera corpus, trained the rail-isolated V4 estimator from my V3 checkpoint, and integrated its hash-bound runtime.

Validation performed by me: I ran the pinned ten-case language evaluation, the Validation and post-selection regression checks, and the preregistered one-shot Final Test reported in chapter 11.

9.1 Building the English task interface

9.1.1 Local model selection and configuration

Qwen2.5-1.5B-Instruct is the pretrained model selected for the interface [44]. In my pipeline, it proposes structured task fields from English requests. I ran the quantised GGUF file distributed in the official model card [32] locally on the CPU through `llama-cpp-python` v0.3.16, a Python binding for `llama.cpp` [3, 10]. I did not fine-tune the model. The exact deterministic inference configuration and artefact identity are indexed through appendix A.

Without fine-tuning, a versioned prompt constrains the model to one `SemanticParseEnvelope` JavaScript Object Notation (JSON) object over permitted task fields, identities, rails, stations and slots; it excludes PDDL, plan steps and device or shuttle commands.

9.1.2 From model output to a confirmed task

A rule-based extractor records explicit facts (for example, shuttle colour, rail or slot), Qwen proposes intent fields, and fixed-precedence fusion prevents the proposal from overwriting explicit or confirmed context. The resulting non-executable `TaskGoalDraft` is clarified, domain-validated and confirmed before an immutable `TaskGoal` is issued.

For the indirect request “Could you get a loaded shuttle over to the right slot 4?”, the saved record shows the model supplying `goal_type=transport` and deterministic fusion/validation constraining side, payload and destination. The ten-case evaluation is reported in section 11.3.1.

9.2 Generating the Room 315 visual corpus

9.2.1 Paired-camera corpus and partitioning

The generator enumerated all $2^8 - 1 = 255$ non-empty subsets of identities L1–L4/R1–R4 and produced eight geometrically varied scenes per subset by changing valid rail positions, payload states and switches. After each Gazebo scene settled, it captured one synchronised image per overhead camera: 2,040 paired observations, or 4,080 RGB files. Figure 9.1 shows one input.

One scene is one labelled paired-camera record; a visible-shuttle target is one present identity with complete supervision and is the denominator used for the per-shuttle metrics.

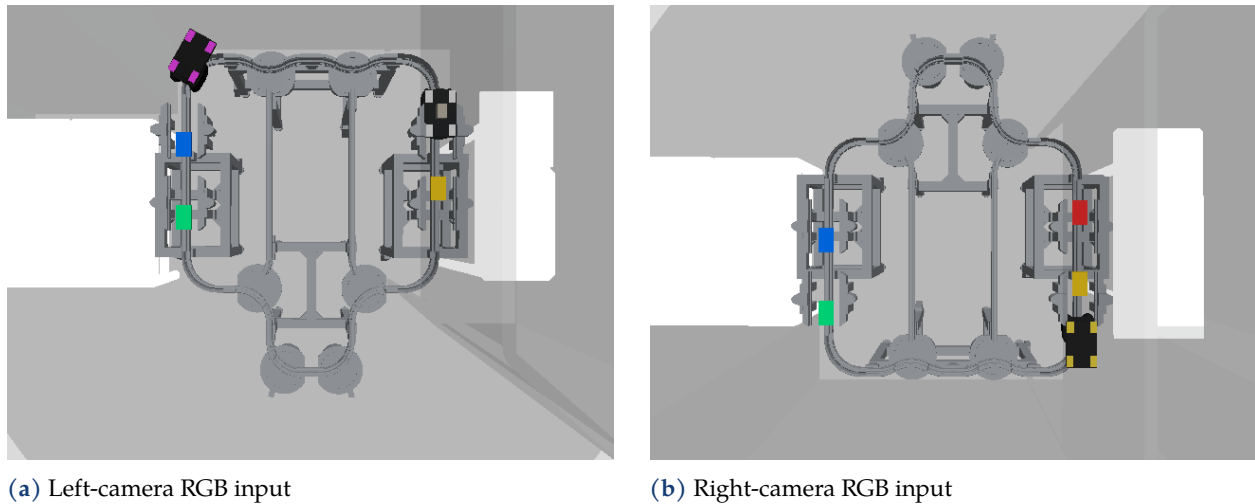


Figure 9.1: The two synchronised RGB images form the complete learned input for one scenario. Simulator state is used to construct labels in a separate file and is never appended to the image tensor.

Gazebo identity, rail coordinate, payload state and projected box supply labels stored separately from image references. Only paired RGB images are model features; depth, controller state, point clouds and other oracle metadata remain label-generation data.

Historical grouped protocol. The predecessor V3 experiment kept all variants of a presence configuration in one partition. Its Validation selected epoch 14, and its Test was consumed once after selection on 30 July 2026; it is therefore historical evidence, not the current Final Test. Neither historical 256-scene partition directly enters V4, although V3’s Validation-selected backbone provides indirect initialisation lineage. Current loaders denylist both historical partitions. Their split counts are summarised in appendix B, while the audit record is indexed through appendix A.

9.2.2 Current training, Validation and regression-set roles

The current pool combines 1,528 legacy grouped-training scenes with 4,000 training-only hard cases. Weighted sampling emphasises rare rail–segment cells, boundaries, switch zones and multi-blocker scenes. These cases were generated before Final Test access rather than mined from its errors.

The separate hard-case Validation (512 scenes; 2,254 present targets) selected V4, drove patience and supplied temperature/segment-threshold fitting. Only after selection, the 256-scene post-selection regression set (1,085 targets) checked the frozen candidate for degradation; it affected neither selection nor calibration and is not Test evidence. Accessing the current Final Test remained prohibited throughout development; it supplies the principal evidence below.

The labelled partitions were released only after the immutable attempt had completed; this later publication changes availability, not their experimental roles. Appendix A identifies the release.

9.3 Designing and training the visual-state estimator

The *visual model* is the learned network; the *visual-inference runtime* adds preprocessing and observation validation; the *closed-loop runtime* further adds the task gateway, planner, supervisor and controllers.

Evolution of the visual pipeline. The early versions primarily evolved the data contract: V1 established paired-camera inputs while keeping simulator-derived labels outside the model input, and V2 added continuous along-rail localisation. V3 fixed the representation to the eight known shuttle identities, resized both cameras to 224×224 , concatenated global features from a shared ImageNet-pretrained ResNet-18 and predicted 200 values through one head. Review of the hard-case behaviour exposed three mismatches between that contract and the known camera geometry and topology. Table 9.1 maps each diagnosis to the corresponding V4 decision. V4 is the active visual model used by the evaluated runtime; its results are in section 11.2.

Table 9.1: V3 limitations and the corresponding V4 design responses.

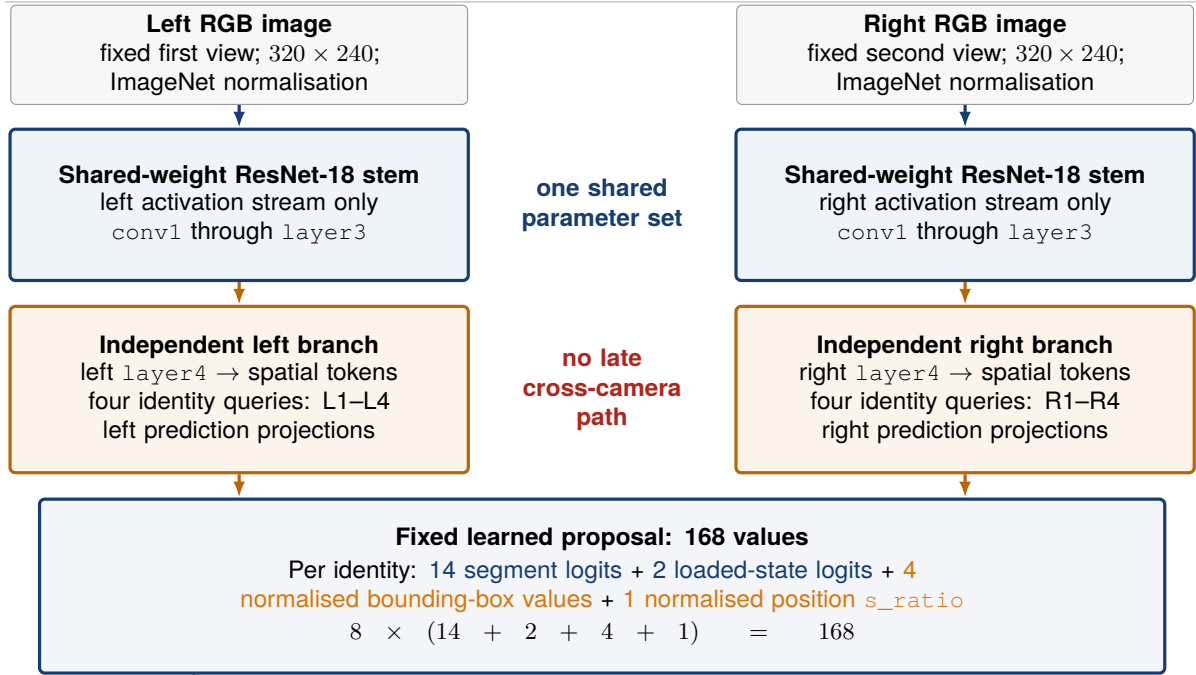
Design concern	V3 observation	V4 response
Image geometry	Squaring the native 4:3 images to 224×224 changed their proportions	Aspect-preserving 320×240 preprocessing retains the camera geometry
Rail interaction	Concatenated global features allowed one camera to influence the other rail's prediction	Low-level weights remain shared, but each rail has an independent late branch and identity queries
Learned-state scope	One 200-value head also predicted side, metric position and segment length, although topology already fixes them	The model learns 168 uncertain visual values; identity and topology derive the remaining quantities deterministically

9.3.1 Rail-isolated 168-value learned contract

Each 4:3 image is resized to 320×240 , ImageNet-normalised and processed separately by shared ResNet-18 weights through `layer3`. Each rail then has its own `layer4` and four identity-query heads; no camera feature enters the other rail's late branch.

For each of L1–L4/R1–R4, the model outputs 14 segment scores, two loaded-state scores, a four-value normalised box and one within-segment ratio: 21 values per identity and 168 in total. Identity and the decoded topology then determine side, segment length and metric position; these known quantities are not learned. The output composition is summarised in appendix B; its exact ordering and diagnostic compatibility adapter remain in the model records indexed by appendix A.

A RAIL-ISOLATED LEARNED ESTIMATOR



B DETERMINISTIC DERIVATION AND RUNTIME GATE

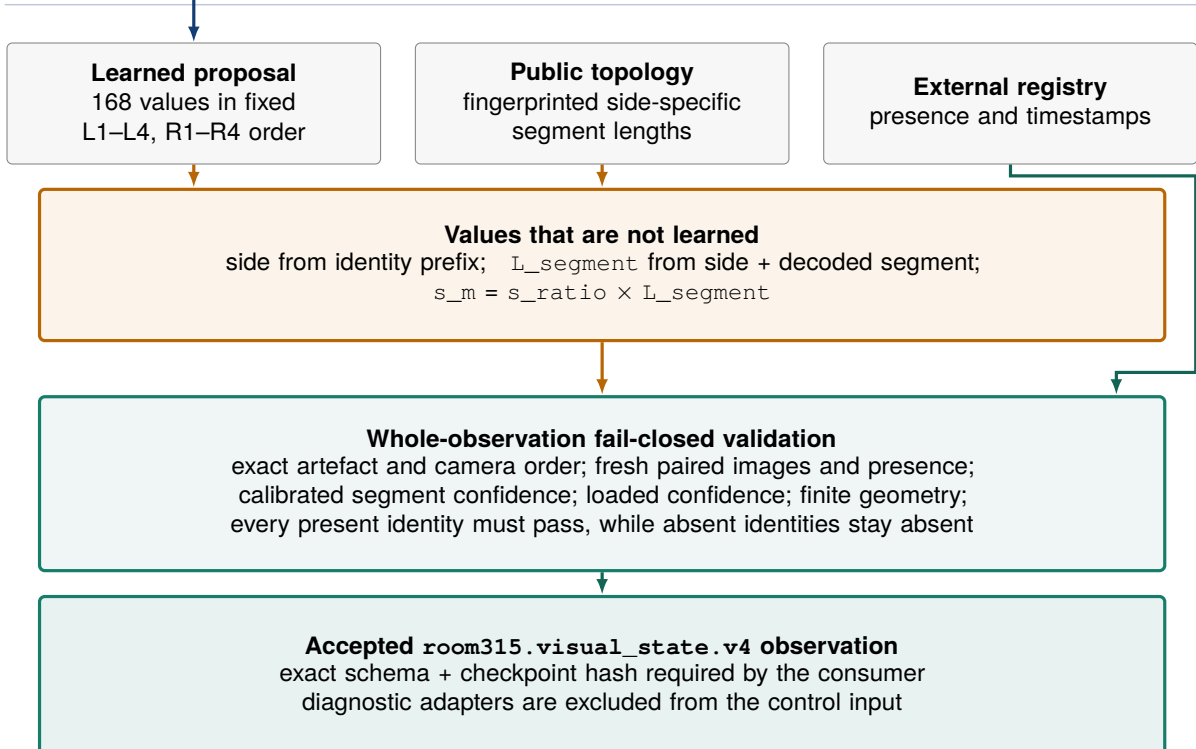


Figure 9.2: The model shares low-level ResNet-18 weights through layer3, then keeps layer4 and the four-query prediction head independent for each rail. The model produces 168 learned values; side, segment length and s_m are derived deterministically before whole-observation validation.

Presence is an independent input, not a prediction. Training masks any identity that is absent or lacks a required target. At runtime, every present identity must pass artefact, camera-order, freshness, vocabulary, numeric and confidence checks; any failure rejects the whole observation rather than publishing a partial state.

Table 9.2: Learned objectives and their role in the selected V4 training run.

Output	Loss weight	Role
Public segment	Cross-entropy, 4	Primary discrete location used by symbolic planning
Within-segment ratio	Smooth-L1, 2	Continuous longitudinal refinement within the selected segment
Loaded state	Cross-entropy, 1	Payload constraint used for grounded task selection
Bounding box	L1/generalised intersection over union (GloU) [33], 1	Identity-localised visual support and diagnostic overlap
Topology distance	Graph distance, 0.25	Penalises mistakes according to their distance on the known route map

9.3.2 Training and checkpoint selection

The V1–V3 visual-state development cycle was trained on the KAIROS cluster operated by CALMIP, using an NVIDIA GH200 GPU. The final V4 training, fine-tuning and evaluation were performed locally on an NVIDIA RTX 3080 Laptop GPU.

Verified predecessor weights initialised the shared stem through `layer3` and each independent `layer4`; no joint head transferred, and rail-branch parameters, batch-normalisation running statistics (BatchNorm buffers) and query heads were optimised independently.

The newly initialised query heads were trained before progressively exposing the deeper rail branches and shared stem. This reduced the chance that an unstable new head would immediately disturb inherited features. Appearance augmentation was allowed, whereas camera swaps and side-breaking flips were forbidden because they would violate the rail contract. The exact optimisation, sampling and evaluation controls are collected in table 9.3.

Table 9.3: Selected V4 optimisation, selection and evaluation controls.

Control	Frozen setting	Purpose
Optimisation	AdamW [17]; learning rates 3×10^{-4} for the heads, 10^{-4} for <code>layer4</code> and 2×10^{-5} for shared <code>layer3</code> ; weight decay 10^{-4}	Separate rates protect inherited low-level features
Staged unfreezing	Heads from epoch 1, independent <code>layer4</code> from epoch 2 and shared <code>layer3</code> from epoch 4	Introduce capacity from task-specific to shared features
Epoch definition	8,000 records sampled with replacement: 4,000 legacy and 4,000 hard-case records; batch 24 and mixed precision	Keep both pools present while emphasising rare cells and boundaries
Candidate selection	Planning score below, then worst rail–segment recall, then lowest 95th-percentile correct-segment position error; patience four	Choose one checkpoint from Validation using criteria fixed before training
Evaluation firewall	Seed 31520260811; post-selection regression excluded from selection and calibration; Final Test unavailable until freeze	Prevent later evaluation data from influencing the selected model

Macro F1 averages the class-wise harmonic mean of precision and recall, giving each segment class equal weight. The fixed Validation planning score was

$$0.30F_{1,\text{macro}} + 0.15A_{\text{min side}} + 0.20R_{\text{worst cell}} + 0.20A_{\text{joint},0.05} + 0.075A_{\text{switch}} + 0.075A_{\text{boundary}}.$$

All 12 epochs completed, the procedure selected epoch 11 and the patience condition was not reached. The regression set could reject this frozen candidate but could neither choose a checkpoint nor tune it; the Final Test remained unavailable throughout.

Validation also fitted the segment-score calibration and online acceptance floor. Loaded-state score remained uncalibrated. The runtime binds checkpoint, architecture, preprocessing, camera order and topology fingerprint before becoming ready. Table 11.2 separates these online checks from labelled offline tolerances; appendix A binds the exact selected-run artefacts.

9.3.3 Evidence after the V3-to-V4 redesign

The architectural diagnosis in the preceding sections was followed by a large change on the retained hard-case Validation comparison (table 9.4). The comparison is useful evidence of the complete redesign and current training protocol, but it is not a controlled ablation: resizing, branch isolation, output definition, data weighting and training schedule changed together. It therefore does not prove how much of the improvement came from any one change.

Table 9.4: Development comparison before and after the V4 redesign.

Hard-case Validation measure	V3	V4	Change
Public-segment top-1 accuracy	51.6415%	99.9113%	+48.2698 points
Loaded-state accuracy	66.1934%	99.8669%	+33.6735 points

The comparison was development evidence: it justified carrying V4, rather than V3, into the frozen qualification sequence, but it was not counted as Final-Test performance. I stopped redesigning only after the selected epoch also met the predeclared Validation and post-selection regression gates and its artefact, preprocessing and topology identities were frozen. The independent opposite-camera blank/shuffle check later changed own-side V4 outputs by zero at recorded precision, supporting the intended rail isolation. Passing these gates defined the decision to test the model; they were selection controls rather than a guarantee of future predictions.

9.3.4 Preregistered Final Test

Before evaluation, I froze the checkpoint, scenario plan, evaluator, thresholds and single-run policy. No Final-Test result could alter training, checkpoint selection, calibration or thresholds. Here, *preregistered* means that the repository controls and hashes were fixed before any Test prediction, not that the protocol was registered externally. The one-shot method, audit and results are reported in section 11.2; their records are indexed through appendix A.

The final AI layer emits only a confirmed task and scene record; deterministic planning and supervision remain responsible for commands. The V3 failures motivated the rail-isolated V4 design, whose frozen evaluation covers only the known fleet and fixed Gazebo cameras (section 11.2).

Chapter 10

From an English Instruction to Shuttle Motion

Chapter guide. I combined the confirmed goal and accepted scene in an observe–plan–supervise–verify loop. Each iteration accounts for Room 315’s one-way routes, occupancy and device states, handles temporary blocker moves and replans from the next observation.

Contribution boundary

Start-of-internship baseline: The starting repository contained no Room 315 PDDL domain or English-to-motion loop. I reused the simulation, rail control and typed contracts that I had developed earlier.

Project and supervisor input: The supervisors reviewed the neuro-symbolic architecture and supplied platform constraints.

Reused foundation: I used PDDL to define symbolic states, actions and goals [19], PlanSys2 to request plans [18], and POPF to search for action sequences [5]; I developed none of them.

My contribution: I defined and implemented the Room 315 PDDL domain and fresh-problem builder, integrated PlanSys2, added deterministic plan translation, and developed the observe–plan–supervise–verify–replan executive.

Validation performed by me: I ran planning, translation and effect-checking tests, twelve closed-loop Gazebo cases and five fault cases reported in chapter 11.

10.1 Runtime architecture

Figure 10.1 shows the loop and the feedback path used while a grounded goal remains incomplete. Interface rules are in chapter 8.

Table 10.1 gives the principal symbolic decisions before the complete runtime sequence. It links a room situation to the conditions that make an action applicable and to the state change predicted by that action.

Table 10.1: Principal decision cases in the Room 315 PDDL domain.

Current situation	Decisive conditions	Applicable action and symbolic result
Exterior-route preparation	Validated side and connected stations; normal-route and device facts	<code>prepare_switches</code> and <code>open_stoppers</code> establish readiness
Direct exact-slot move	Matching source occupancy; free target; ready route; no pending clearance	<code>move_shuttle_to_slot</code> transfers occupancy and records the reservation
Topology or segment-origin route	Known topology block and authorised route to the target	Prepare, then move; a segment-origin action does not assert source-slot occupancy
Blocked required route	Ordered clearance and a capacity-safe holding destination	Relocate one blocker, observe the effect and rebuild the problem
Route normalisation	Clearance complete and reconfiguration allowed	<code>restore_normal_route</code> re-establishes exterior-route readiness
Terminal or inspection state	Target/device facts hold, or inspection is confirmed	Terminal actions predict completion; <code>inspect_state</code> never actuates

I translated each operating rule into predicates, action preconditions and effects. I then tested valid cases and closely related invalid cases. The tests covered both rails, slots, switches, stoppers, shuttles obstructing a route, and interior and exterior paths. A valid state had to produce an allowed step. Missing, stale, contradictory or impossible state had to produce no executable command. After execution, new sensor and controller feedback had to match the expected effect before replanning.

These tests provide evidence for the covered cases; they are not a mathematical proof that the domain contains no error.

10.2 Why the executive uses one-step replanning

A complete POPF sequence is inspectable, but later steps are predictions about a state that has not yet been observed. Executing the whole sequence open-loop would let an occupancy change, failed device update or incomplete arrival make its remaining preconditions obsolete. The executive therefore translates and checks only the first supported action, observes its effect and builds a fresh problem before continuing. This costs additional planner calls, but it keeps the next decision tied to accepted state rather than to an earlier symbolic forecast.

Engineering iteration for blocker staging. An early planning assumption treated several blocker destinations as if they could be allocated from one unchanged observation. Dense configurations exposed the flaw: moving the first blocker changes both occupancy and the capacity-safe destination available to the next one. I changed the design to authorise one relocation at a time, retain the required holding capacity, obtain a fresh observation and then either relocate the next blocker or restore the normal exterior route. Builder and regression cases cover both rails and full-rail distributions; they establish the implemented bounded strategy, not a general deadlock proof.

10.3 Step 1—interpret and confirm the English task

The CLI applies chapter 9, resolves identities and colour aliases, and clarifies missing fields before confirmation publishes `TaskGoal`. Scene-dependent policies remain ungrounded: the gateway selects the shuttle or admissible station slot from current state, while the planner chooses actions.

10.4 Step 2—read the current scene and ground the goal

The hash-bound visual runtime checks schema, preprocessing, camera order, freshness and confidence. Only the gateway can fuse accepted visual locations with presence, device and identity-bearing sensor reports or change `PlanSys2` state. It grounds *any*, *nearest* and station-slot policies against current occupancy; payload selection requires five consecutive accepted observations, after which the identity remains fixed.

10.5 Step 3—build a fresh PDDL problem and plan

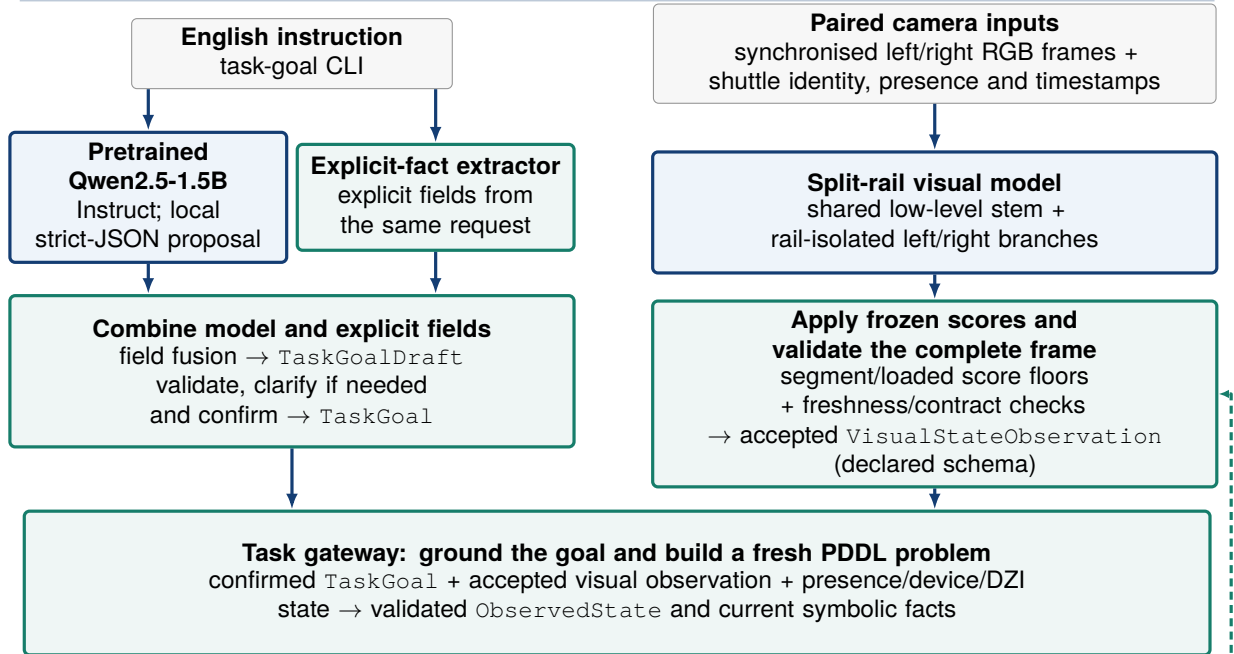
Each cycle turns the grounded goal and latest valid `ObservedState` into a fresh PDDL problem containing shuttle, location, occupancy, payload and device facts. It receives no raw image or text; unknown, stale or conflicting state blocks generation instead of implying free space. Transport actions require validated side, location, occupancy and route facts. The direct, blocked-route, normalisation and terminal cases are summarised in table 10.1; the predicate-level reference remains in the versioned PDDL artefact indexed by appendix A.

POPF minimises symbolic setup and relocation cost, not travel time. The executive uses the `PlanSys2` planner service rather than its action `Executor`, translates the plan, and selects only its first supported checked step. PDDL effects remain predictions: the next cycle always starts from the observed post-action state.

10.6 Steps 4 and 5—execute one step and verify its effect

The translator makes a sparse named `TranslatedPlanStep`; the supervisor checks it before publishing a typed command. The executive then checks the observed effect and replans if incomplete.

A INTERPRET THE REQUEST AND ACCEPT THE SCENE STATE



B PLAN, EXECUTE ONE SAFE STEP, AND CHECK THE RESULT

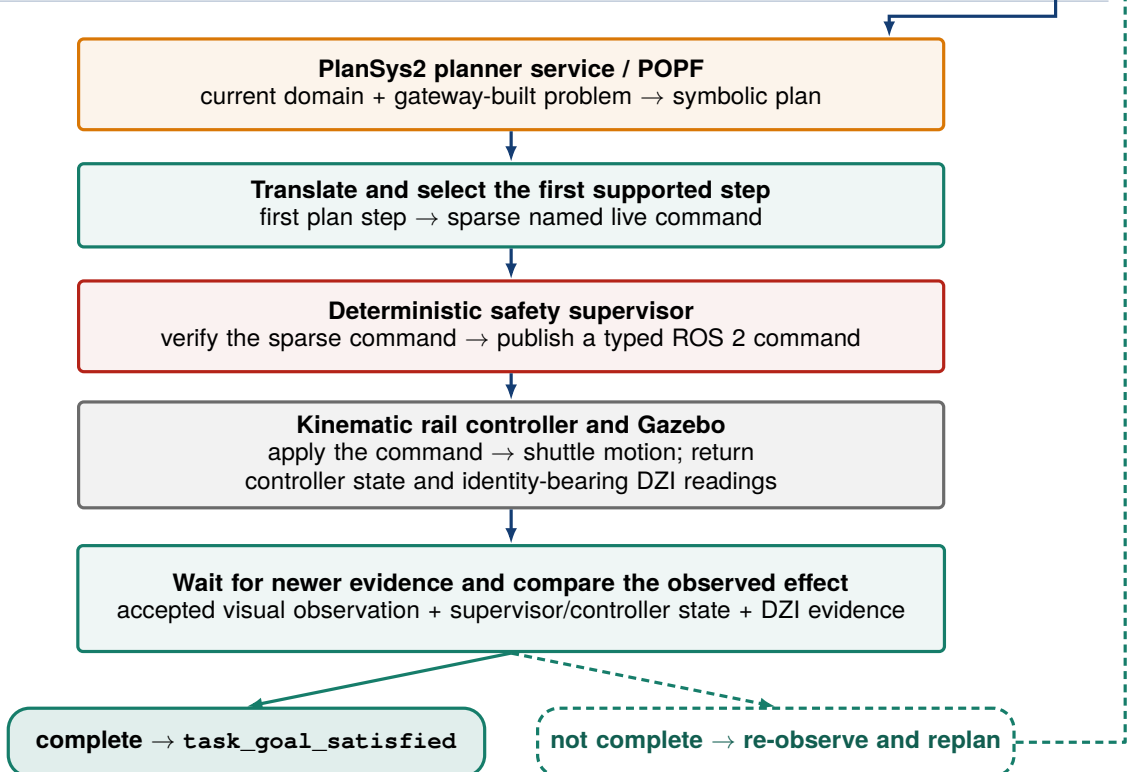


Figure 10.1: Complete layered sequence from task and paired-camera inputs to a verified Gazebo result; the dashed green path denotes observation and replanning while the grounded goal remains incomplete.

Final-slot success requires the DZI/controller evidence in chapter 8; a command alone is never success.

10.7 Case B03 from instruction to arrival

The retained B03 fresh-process run is small enough to expose every boundary from request to terminal evidence (table 10.2). Its before/after camera frames appear in figure 11.1.

Table 10.2: Stage-by-stage record for representative campaign case B03.

Stage	Recorded result
Request and goal	“Move the green shuttle to slot 4 on the right rail”; confirmed as R3 to right slot 4
Current state and plan	R3 observed at slot 3, slot 4 free; POPF returned one supported direct move
Command	Supervised <code>move_shuttle_to_slot</code> , right slot 3 to slot 4, at 0.2 m/s
Effect	Two newer DZI4R reports identified R3; controller reported target slot 4 and <code>DISABLED</code>
Outcome	All postconditions satisfied; <code>task_goal_satisfied</code>

Why this record counts as success. The five rows are not five equivalent success claims. The confirmed request fixes what must be achieved, and the current-state row shows why the returned action was applicable; neither proves that motion occurred. Likewise, the supervisor’s accepted command proves authorisation, not arrival. The effect row is the decisive evidence: two reports newer than the command identify R3 at the requested slot, and the controller independently reports that the target slot was reached and motion was `DISABLED`. Only after those declared postconditions agree does the executive append `task_goal_satisfied`. The trace therefore separates intended effect, symbolic prediction, command authorisation and observed effect instead of equating a planner response or published command with success.

Scope of the example. B03 deliberately uses one direct move so that every software boundary remains visible in a short record. It does not exercise blocker relocation, route restoration or competing occupancy; those cases appear in the twelve-case matrix in section 11.1 and table B.3.

The implemented loop grounds, plans and authorises one action at a time, then replans from the observed effect. The B03 trace shows this contract end to end; the campaign in section 11.1 covers blocker clearance and occupancy changes beyond that direct-move example.

Chapter 11

Experiments and Results

Chapter guide. I evaluated component behaviour separately from complete task execution because the two answer different questions. The evidence covers software, language, perception, faults and closed-loop operation before being mapped to objectives O1–O4.

Contribution boundary

Start-of-internship baseline: The starting repository contained no Room 315 evaluation protocols, campaign runners or evidence archive.

Project and supervisor input: The supervisors supplied scientific direction, platform knowledge and review; project resources supplied platform access, GPU time and simulation time.

Reused foundation: The evaluated system uses external ROS 2, Gazebo, Qwen2.5-1.5B, PlanSys2 and POPE. I implemented the tests with `pytest` [30]; I built the vision pipeline with PyTorch [29] and TorchVision’s ResNet-18 [11, 31]. I developed none of these tools or base models.

My contribution: I designed and implemented the test suites, acceptance criteria, case matrices, runners, metrics, gates, provenance checks and evidence archive.

Validation performed by me: I ran and reviewed every software, language, vision, observation, fault and closed-loop evaluation reported here. After qualification was frozen, I manually approved the Validation-selected V4 checkpoint for runtime; the Final Test did not alter the checkpoint, calibration or thresholds.

11.1 Closed-loop campaign result

11.1.1 Method and acceptance criteria

Before execution, I fixed twelve case IDs, English instructions, expected grounded goals and initial shuttle configurations. Slot goals named one destination; station goals accepted any admissible slot. The runner bound the source, language setup, epoch-11 visual checkpoint, runtime manifests and PDDL domain by hash. It checked source/install parity and started every case in a fresh Gazebo process and ROS Domain ID. Before accepting input, it awaited the world, controllers, cameras, authorised perception, planner, supervisor and gateway.

For each row, I checked the displayed interpretation and entered interactive `yes`. Confirmation was never automated. Initial and final observations had to use the declared schema and checkpoint. They also had to identify every present shuttle’s side, segment and payload correctly and keep $|\Delta s_{ratio}| \leq 0.12$ against the expected scene. A run passed only when the requested terminal state, symbolic postconditions, DZI arrival evidence and stopped-controller state agreed, with clean camera and ROS-bag closure. Case-level paths and outcomes are summarised in appendix B; full declarations and retained records are indexed through appendix A.

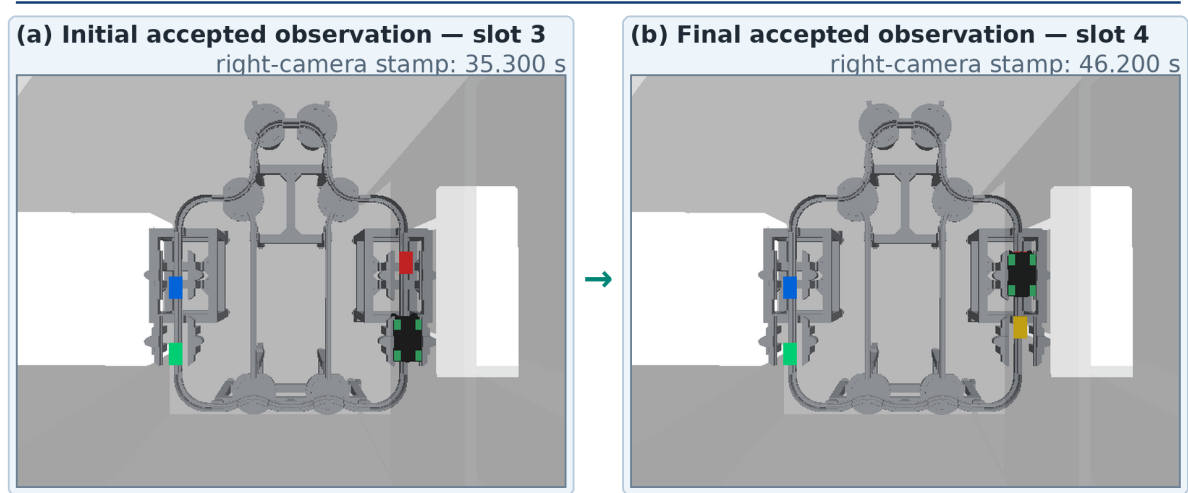
All 12 runs succeeded, all 24 step postconditions were satisfied and the supervisor accepted the resulting 48 low-level proposals. The executive made 24 planner requests and replanned four times after occupancy changes; no run needed an unknown-result retry or safe abort. Every run ended with `task_goal_satisfied`, verified final effects and disabled controllers. The 1,784 schema-compliant observations are recorded ROS messages, not independent accuracy trials. Each scene was executed once, so the result proves the declared Gazebo paths rather than repeated-run reliability.

Table 11.1: Coverage and outcome of the twelve-case closed-loop campaign.

Case group	Cases	Coverage	Result
Direct transfers	B01–B08	Both rails, all eight shuttle identities, colour aliases, explicit identities and station goals	8/8 passed
Payload selection	P01–P02	Loaded-nearest selection plus blocker clearance on each rail	2/2 passed
Multi-shuttle routes	M01–M02	Four-step clearance and transfer route on each rail	2/2 passed

Figure 11.1 gives a representative before/after comparison; the corresponding DZI and controller records establish the terminal state.

V4 campaign case B03 · R3 transport · slot 3 → slot 4



DZI3R/DZI4R and stopped-controller records certify the slot transition.

Figure 11.1: Frames selected by the initial and final accepted observations in the B03 ROS bag. Identity-bearing DZI3R/DZI4R and controller records establish R3’s arrival at slot 4 and the final `DISABLED` controller state.

11.2 Final Test and runtime qualification

The Final Test evaluates the learned outputs in figure 9.2; topology-derived side, segment length and metric position are excluded. Validation is the 512-scene selection set, while the earlier 30 July Test is consumed and is not the Final Test below.

11.2.1 One-shot Final Test method

Frozen epoch 11 ran once on 1,040 scenes, 2,080 images and 4,680 visible targets. Coverage selection used preregistered identifiers and aggregate support counts only; no model predictions or individual image or label contents were inspected. No training, model selection or calibration refit followed. The protocol checked disjoint scene IDs, generation-payload fingerprints, image bytes and complete label rows. It passed 76 preregistered model/support gates and four runtime-threshold gates. A gate is one predeclared Boolean acceptance check, not an independent trial. The 76 gates covered global, rail, segment, target-zone, position-bin, scene-stratum and camera-counterfactual criteria. The four runtime gates covered minimum segment-confidence coverage, loaded-state coverage, joint-confidence coverage and selective segment accuracy. The checkpoint, sealed dataset, controls

and complete gate and stratum records are indexed through appendix A; per-rail summaries are retained in appendix B.

Online checks decide whether an unlabelled prediction may enter the runtime; offline labelled tolerances measure its error afterwards. With estimate $\hat{s}_{ratio}$ and label s_{ratio}^{label} , the reported error is $|\Delta s_{ratio}| = |\hat{s}_{ratio} - s_{ratio}^{label}|$. The distinct uses are summarised in table 11.2.

Table 11.2: Online runtime checks and offline labelled tolerances.

Check	Interpretation
Online runtime checks	
Segment score ≥ 0.9139	Temperature-scaled maximum over 14 segments. Below the threshold, the whole paired observation is withheld; passing does not guarantee correctness.
Loaded-state score ≥ 0.5	Uncalibrated binary maximum. Because this maximum is always at least 0.5, the check is non-selective; 0.5 does not mean 50% correctness.
$d_{slot} = \hat{s}_{ratio} - s_{ratio}^{slot} \leq 0.12$	Distance to the configured slot anchor, used for occupancy. It is neither confidence nor comparison with a hidden label.
Offline labelled tolerances	
Correct segment and $ \Delta s_{ratio} \leq 0.05$	Strict ground-truth diagnostic: error within 5% of segment length, not 5 cm.
Correct segment and $ \Delta s_{ratio} \leq 0.12$	Planning/campaign-checker tolerance for the expected scene: error within 12% of segment length, not 12 cm. Although both use 0.12, this is not the online slot check.

Final arrival uses neither offline position tolerance: it requires command-linked DZI and controller-state evidence as described in chapter 8.

Here, top-1 accuracy counts a target when the highest-scoring segment is its labelled segment.

Table 11.3: Global essentials from the one-shot Final Test.

Metric	Global result
Public-segment top-1 accuracy	99.9359%
Loaded-state accuracy	99.9145%
Joint segment and $ \Delta s_{ratio} \leq 0.05$	90.8974%
Joint segment and $ \Delta s_{ratio} \leq 0.12$	99.0385%
Correct-segment longitudinal mean absolute error (MAE) / 95th percentile	0.01573 m / 0.05850 m
Mean bounding-box intersection over union (IoU)	0.83400

V4 almost always recovered the discrete segment and loaded state needed by the planner. Fine longitudinal localisation was weaker: 90.8974% met the strict 0.05 diagnostic, whereas 99.0385% met the 2.4-times wider planning tolerance. The worst segment-recognition cell was left-A14 (94.6429% recall on 56 targets); the weakest strict-position cell was left-A1E (61.9469% at 0.05, but 98.2301% at 0.12 on 113 targets). Detailed per-rail, stratum and support results remain in the Final-Test records indexed through appendix A; the per-rail summary is in appendix B. These fixed-camera results do not establish physical-camera, unseen-condition or complete task performance.

At the online segment floor, 4,678/4,680 targets were retained: two abstained, while two different incorrect segment predictions passed. The loaded-state floor retained all targets, including four incorrect decisions. Confidence therefore controls acceptance rather than truth. Blanking or shuffling the opposite camera changed own-side outputs by zero at recorded precision, whereas swapping camera order reduced segment accuracy to 7.7991%; this supports rail isolation and the enforced camera order.

11.2.2 Development and observation checks

Validation passed 76/76 gates and selected the checkpoint and segment-score calibration; the small post-selection regression set passed 74/74 gates and could only reject the frozen candidate. Neither is independent evidence, so the one-shot Final Test remains the reported evaluation. Full development metrics are indexed through appendix A; appendix B retains a compact summary.

Observation-only checks disabled PlanSys2 mutation and actuation. Seven scenes completed 53/53 reobservations at the 0.12 planning tolerance; 150/169 identity checks also met the stricter 0.05 diagnostic. All eight injected visual-input faults were rejected, and a dense-scene dry run produced 10/10 matching reobservations. The isolated same-frame shadow accepted 52/55 pairs without wrong-side identities or reported errors; the aggregate record does not explain the remaining three rejections. Consequently, this result supports the observation boundary but not motion or physical-camera performance.

11.2.3 Closed-loop fault qualification

Five faults were injected in fresh Gazebo processes. The case matrix in table 11.4 makes the negative oracle explicit: the expected result is a safe refusal or stop, never task completion.

Table 11.4: Closed-loop fault cases, expected response and recorded outcome.

Case	Injected fault	Predeclared safe response	Recorded outcome
F01	Corrupt authorisation	Reject before motion; no success record	Passed; no false completion, controllers <code>DISABLED</code>
F02	Invalid runtime manifest	Refuse runtime readiness and motion	Passed; no false completion, controllers <code>DISABLED</code>
F03	Missing planner service	End safely without publishing a motion command	Passed; no false completion, controllers <code>DISABLED</code>
F04	Emergency stop during a long move	Invoke <code>stop_all</code> , latch the stop and withhold completion	Passed; no false completion, controllers <code>DISABLED</code>
F05	Loss of right-sensor feedback	Do not certify arrival; stop or abort safely	Passed; no false completion, controllers <code>DISABLED</code>

Across the five cases, all 5/5 criteria passed and all 424 observations used the active schema. A subsequent B01 smoke run completed its step and postcondition with 88 matching observations. Together, these records satisfy the declared fault-response criteria.

11.3 Software, language and full-floor checks

Three named suites checked the implementation before the integrated runs. The contract/executive suite passed 190/190 checks, the transport suite 96/96 and the complementary operator-capability suite 42/42. They cover geometry, topology, identity, devices, reservations, language contracts, planning, supervision and effect checks, with protective and fault cases. Because a few files overlap, the suite counts are reported separately rather than summed; their commands and logs are indexed in appendix A.

11.3.1 Language-interface evaluation

The fixed ten-case English corpus contained four supported goals, three incomplete or ambiguous requests, two inconsistent or invalid requests and one cancellation. All 10/10 final outcomes matched: valid goals produced the expected constraints, uncertain requests requested clarification, invalid requests were rejected and cancellation produced no task. The configured local backend was called once for each of the nine non-control cases; eight envelopes passed the strict schema, while the predicted invalid slot 9 was rejected. This tests the implemented Room 315 contract, not open-vocabulary or multilingual language understanding.

11.3.2 All-robot launch check

An isolated `robots:=all` cold start contained all six configured models. Each exposed namespaced joint command and feedback topics and produced a fresh `JointState`; both TIAGo variants also exposed independent base-velocity topics, with no declared error pattern before shutdown. This one check supports launch composition and namespace separation, not simultaneous motion or repeatability. The complete record and operator procedures are indexed in appendix A.

11.4 Review of the internship objectives

Table 11.5 returns to O1–O4 and links each objective to its principal evidence.

Table 11.5: Assessment of the four internship objectives.

Objective	Main result	Evaluation
O1—simulation platform	Third-floor CAD and room furnishings designed by me in Autodesk Inventor; supplied rail-cell and attributed robot assets integrated into full-floor and Room 315 scenes; selectable launch profiles retained	Static geometry/SDF checks and one six-robot cold-start launch and interface check
O2—multi-shuttle transport	Calibrated Hermite rail geometry, explicit switch-dependent topology, eight stable identities, typed devices, reservations, headway checks and bounded blocker staging on both rails	96/96 named transport checks and route-clearance cases in the integrated campaign
O3—contracts and supervision	Versioned command, state and effect contracts; freshness and consistency gates; supervisor rejection paths; command-linked post-step effect checks	190/190 focused checks plus positive and fault campaign records
O4—neuro-symbolic workflow	Rail-isolated perception and checked execution from English requests; frozen Validation-selected V4 manually approved for the Gazebo runtime without Final-Test-driven tuning	10/10 language cases, 1,040-scene one-shot Final Test, 12/12 positive and 5/5 fault runs plus observation-only qualification

Because the evaluations measure different properties, their counts are reported separately. Each objective is considered achieved only within the limits of the corresponding tests. Thus the launch check supports configuration and feedback, the Final Test supports synthetic-camera perception, and the closed-loop campaign supports the twelve declared tasks. Repeated operation was not evaluated.

Overall, the results support O1–O4. Fine shuttle localisation remains the weakest result: 90.8974% at the 0.05 tolerance versus 99.0385% at 0.12, while three of the 55 shadow comparisons remain unexplained. The evidence remains Gazebo-only and neither authorises physical deployment nor establishes machinery-safety certification.

Chapter 12

Limitations and Perspectives

Although Chapter 11 establishes controlled evidence, transfer, repeatability and recovery remain unresolved. The discussion below links each limitation to its engineering consequence and to the evaluation needed before broader claims can be made.

12.1 Measured limits

Simulation and operation. The six-robot check was one cold start, not a simultaneous-motion or repeatability study. Room 315 uses a kinematic shuttle backend that omits torque, slip, braking, play and payload inertia. Positive and fault cases were each run once; long outages, profile switching, longer wait-for cycles, temporal non-progress and device acknowledgement correlated to a specific command remain untested.

Perception and data. The visual results concern the known fleet, fixed simulated cameras and generated scene families. Scene IDs, generation-payload fingerprints, image bytes and complete label rows are disjoint, but a post-hoc audit found repeated target-state, trajectory and geometry support across development partitions. This is not sample reuse, yet it means the evidence is not independent of the underlying synthetic distribution. The Final Test therefore says nothing about new hardware, lighting, textures or installation conditions. Identity presence and rail side also come from configured registries rather than learned outputs.

The segment score is Validation-calibrated, the loaded-state score is not a calibrated probability, and the labelled 0.05/0.12 tolerances are offline measures rather than runtime confidence. Their relationship to the separate online slot-radius rule is fixed in table 11.2. The unexplained 52/55 shadow acceptance is an additional observation-level limit.

Task and interface scope. The twelve command-producing cases cover selected valid scenes only; neither the Final Test nor the observation studies execute a task. English evaluation uses ten selected requests, so open-vocabulary and multilingual behaviour are unknown. Finally, CLI risk labels are heuristics and the subscriber assumes a trusted ROS graph; multi-user use would require authenticated sources or graph isolation.

12.2 Priority next steps

Evaluation order and stopping rule. The completed Final Test remains a fixed reference and must not become a tuning set. I would first create the domain-shift data and repeat the existing observation and campaign protocols unchanged. A failure would open a new development cycle evaluated on Validation and the post-selection regression set; it would not justify modifying the completed Test result. Physical deployment is governed by the separate trial gate defined below.

Table 12.1: Priority experiments for extending the evidence.

Priority	Engineering task	Question answered
P0	Create a separate visual-shift set with varied cameras, lighting, textures and sensor noise	Does perception transfer beyond the completed synthetic distribution without tuning on the Final Test?
P0	Repeat the seven-scene observation review across declared seeds	How variable are acceptance and localisation in sparse, dense and blocker scenes, including the current 52/55 shadow result?
P0	Inject timed camera, presence, planner and node-restart faults in the live ROS graph	Can the runtime recover while preserving fail-closed behaviour?
P0	Repeat the positive and fault campaigns across controlled seeds	How variable are completion, abstention, recovery and controller stop?
P1	Correlate switch/stopper acknowledgements with commands and extend reservation/deadlock cases	Do devices and multiple shuttles remain consistent under delayed feedback and longer conflicts?
P1	Mirror the releases and perform a clean-machine restore and replay	Can MFJA recover the exact source, data and evidence independently?

12.3 Continued use at MFJA

The handover remains suitable for Gazebo teaching and research: students can launch selected robots, inspect interfaces, alter rail scenes, generate PDDL and trace a supervised task. Alternative planners, perception checkpoints or grounding methods can reuse the fixed cases without changing controller or arrival contracts. New model selection must use Validation; the small post-selection regression set remains a development check, and new visual conditions require a separate domain-shift evaluation rather than reuse of the completed Final Test. Here, domain shift means a change in the visual-data distribution, not a change to the PDDL action domain or ROS Domain ID.

Gate before a physical trial. Simulation qualification would be a prerequisite, not deployment authority. A future physical trial would also require an independent machinery-risk review, an explicit mapping to the platform’s certified emergency functions, an authenticated command owner, and a low-speed commissioning protocol with predefined abort criteria. None of those controls is supplied by the current Gazebo supervisor, so a successful simulation replay alone must not be used to authorise hardware motion.

The completed Final Test should remain unchanged. Future experiments should examine new visual conditions, repeated runs, live faults and concurrent shuttle operation.

Chapter 13

Conclusion

MFJA needed a reproducible environment that could connect its heterogeneous third-floor robots to checked task execution.

In Phase 1, I designed the third-floor CAD and the furnishings of Rooms 314 and 315 in Autodesk Inventor. I then integrated the supplied rail cells and attributed robot assets into Gazebo. In Phase 2, I implemented multi-shuttle transport, versioned interfaces, visual and language grounding, symbolic planning and a supervisor that verifies effects before reporting success. Together, these elements form one chain from task-level English to verified simulated effects, rather than separate demonstrations. Configuration, automated tests, retained evidence and run instructions form the handover to MFJA.

The declared evaluation succeeded in all 10 language cases and all 12 isolated request-to-motion runs. It also succeeded in all 5 closed-loop fault cases. The one-shot Final Test evaluated 4,680 targets and passed all 80 predeclared gates. Segment classification was 99.9359%; the stricter joint segment/position result was 90.8974% at $|\Delta s_{ratio}| \leq 0.05$. The discrete facts supplied to planning were therefore strong, but fine position along a rail remained the clearest measured weakness. The retained evidence supports O1–O4 within the stated scope. Table 11.5 maps each objective to its principal evidence.

These claims apply to fixed simulated cameras, known shuttles, kinematic rail motion and mostly one execution per scene. They do not establish physical transfer, simultaneous six-robot motion, repeated-run reliability or certified safety. The selected runtime is Gazebo-only and starts with command execution disabled. The priority extensions are a genuinely shifted visual set, repeated seeded campaigns, live-fault recovery and stronger concurrency tests.

For MFJA, the outcome is a reusable platform for teaching, integration and controlled planning or perception comparisons. Working under platform constraints required me to turn a broad request into testable contracts and revise designs when weaknesses appeared. The main professional lesson for me was learning to justify design choices with evidence, discuss them with my supervisors and state clearly what the results do and do not demonstrate.

Bibliography

- [1] AIP-PRIMECA Occitanie. MFJA 3rd floor Gazebo simulation, 2026. URL https://github.com/aip-primeca-occitanie/mfja_3rd_floor_gz. Project repository; report-specific release and revision identities are recorded in Appendix A; Accessed 18 August 2026.
- [2] Autodesk. Autodesk Inventor: 3d CAD software for mechanical design, n.d. URL <https://www.autodesk.com/products/inventor/overview>. Accessed 18 August 2026.
- [3] Andrei Betlen and llama-cpp-python contributors. llama-cpp-python: Python bindings for llama.cpp, 2025. URL <https://github.com/abetlen/llama-cpp-python/tree/v0.3.16>. Version 0.3.16, revision c37132b; Accessed 18 August 2026.
- [4] colcon contributors. colcon: Collective construction, n.d. URL <https://colcon.readthedocs.io/en/released/>. Accessed 18 August 2026.
- [5] Amanda Jane Coles, Andrew I. Coles, Maria Fox, and Derek Long. Forward-chaining partial-order planning. In *Proceedings of the Twentieth International Conference on Automated Planning and Scheduling*, pages 42–49, 2010. doi: 10.1609/icaps.v20i1.13403.
- [6] Dassault Systèmes. SOLIDWORKS 3d CAD, 2023. URL https://www.solidworks.com/sites/default/files/2023-09/SOLIDWORKS_3D_CAD_Datasheet.pdf. Accessed 18 August 2026.
- [7] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. ImageNet: A large-scale hierarchical image database. In *2009 IEEE Conference on Computer Vision and Pattern Recognition*, pages 248–255, 2009. doi: 10.1109/CVPR.2009.5206848.
- [8] Timothy Duggan, Pierrick Lorang, Hong Lu, and Matthias Scheutz. The price is not right: Neuro-symbolic methods outperform VLAs on structured long-horizon manipulation tasks with significantly lower energy consumption. In *Proceedings of the 2026 IEEE International Conference on Robotics and Automation (ICRA)*, Vienna, Austria, June 2026. URL <https://hri-lab.tufts.edu/publications/dugganetal26icra.pdf>. Paper Tu2I.336; author manuscript; Accessed 18 August 2026.
- [9] Foxglove. MCAP format specification, n.d. URL <https://mcap.dev/spec>. Accessed 18 August 2026.
- [10] Georgi Gerganov and llama.cpp contributors. llama.cpp: LLM inference in C/C++, 2023. URL <https://github.com/ggml-org/llama.cpp>. Accessed 18 August 2026.
- [11] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 770–778, 2016. doi: 10.1109/CVPR.2016.90.
- [12] Brian Ichter, Anthony Brohan, Yevgen Chebotar, Chelsea Finn, Karol Hausman, Alexander Herzog, Daniel Ho, Julian Ibarz, Alex Irpan, Eric Jang, et al. Do As I Can, Not As I Say: Grounding language in robotic affordances. In *Proceedings of the 6th Conference on Robot Learning*, volume 205 of *Proceedings of Machine Learning Research*, pages 287–318. PMLR, 2023. URL <https://proceedings.mlr.press/v205/ichter23a.html>. Accessed 18 August 2026.
- [13] INSEE. NAF rev. 2, subclass 85.42Z: Higher education, 2025. URL <https://www.insee.fr/fr/metadonnees/nafr2/sousClasse/85.42z>. Accessed 18 August 2026.
- [14] INSEE. Avis de situation au répertoire SIRENE: Université de Toulouse, SIRET 938 271 392 00012, 2026. URL <https://api-avis-situation-sirene.insee.fr/identification/pdf/93827139200012>. Official SIRENE record; Accessed 18 August 2026.
- [15] International Organization for Standardization. ISO 23247-1:2021 automation systems and integration—digital twin framework for manufacturing—part 1: Overview and general principles, October 2021. URL <https://www.iso.org/standard/75066.html>. Accessed 18 August 2026.
- [16] Nishanth Kumar, William Shen, Fabio Ramos, Dieter Fox, Tomás Lozano-Pérez, Leslie Pack Kaelbling, and Caelan Reed Garrett. Open-world task and motion planning via vision-language model generated constraints. *IEEE Robotics and Automation Letters*, 11(3):3366–3373, 2026. doi: 10.1109/LRA.2026.3656799. URL <https://doi.org/10.1109/LRA.2026.3656799>. Accessed 18 August 2026.
- [17] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019. URL <https://openreview.net/forum?id=Bkg6RiCqY7>. Accessed 18 August 2026.
- [18] Francisco Martín, Jonatan Ginés Clavero, Vicente Matellán, and Francisco J. Rodríguez. PlanSys2: A planning system framework for ROS2. In *2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 9742–9749, 2021. doi: 10.1109/IROS51168.2021.9636544.
- [19] Drew McDermott, Malik Ghallab, Adele Howe, Craig Knoblock, Ashwin Ram, Manuela Veloso, Daniel Weld, and David Wilkins. PDDL—the planning domain definition language. Technical Report CVC TR-98-003/DCS TR-1165, Yale Center for Computational Vision and Control, October 1998. URL <https://ipc08.icaps-conference.org/deterministic/data/mcdermott-et-al-tr-1998.pdf>. Version 1.2; Accessed 18 August 2026.
- [20] NVIDIA Corporation. CUDA C++ programming guide, 2025. URL <https://docs.nvidia.com/cuda/archive/12.8.1/cuda-c-programming-guide/index.html>. CUDA Toolkit 12.8 Update 1 archive, matching the recorded CUDA 12.8 replay environment; Accessed 18 August 2026.
- [21] Open Robotics. Getting started with Gazebo—Gazebo Harmonic documentation, 2026. URL <https://gazebo-sim.org/docs/harmonic/getstarted/>. Undated online documentation; the year field records the access year; Accessed 18 August 2026.
- [22] Open Robotics. Use ROS 2 to interact with Gazebo, 2026. URL https://gazebo-sim.org/docs/harmonic/ros2_integration/. Undated online documentation; the year field records the access year; Accessed 18 August 2026.

- [23] Open Robotics. Interfaces—ROS 2 Jazzy documentation, 2026. URL <https://docs.ros.org/en/jazzy/Concepts/Basic/About-Interfaces.html>. Undated online documentation; the year field records the access year; Accessed 18 August 2026.
- [24] Open Robotics. Integrating launch files into ROS 2 packages—ROS 2 Jazzy documentation, 2026. URL <https://docs.ros.org/en/jazzy/Tutorials/Intermediate/Launch/Launch-system.html>. Undated online documentation; the year field records the access year; Accessed 18 August 2026.
- [25] Open Robotics. Developing a ROS 2 package—ROS 2 Jazzy documentation, 2026. URL <https://docs.ros.org/en/jazzy/How-To-Guides/Developing-a-ROS-2-Package.html>. Undated online documentation; the year field records the access year; Accessed 18 August 2026.
- [26] Open Robotics. Quality of service settings—ROS 2 Jazzy documentation, 2026. URL <https://docs.ros.org/en/jazzy/Concepts/Intermediate/About-Quality-of-Service-Settings.html>. Undated online documentation; the year field records the access year; Accessed 18 August 2026.
- [27] Open Source Robotics Foundation. SDF format specification, n.d. URL <https://sdformat.org/spec/1.11/>. Version 1.11; Accessed 18 August 2026.
- [28] OpenAI. What is ChatGPT: FAQ, 2026. URL <https://help.openai.com/en/articles/12677804-what-is-chatgpt-faq>. Undated official product page; the year field records the access year; Accessed 18 August 2026.
- [29] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems*, volume 32, 2019. URL <https://proceedings.neurips.cc/paper/2019/hash/bdbca288fee7f92f2bfa9f7012727740-Abstract.html>. Accessed 18 August 2026.
- [30] pytest developers. pytest documentation, n.d. URL <https://docs.pytest.org/en/stable/>. Accessed 18 August 2026.
- [31] PyTorch contributors. TorchVision 0.25 ResNet-18 documentation, n.d. URL <https://docs.pytorch.org/vision/0.25/models/generated/torchvision.models.resnet18.html>. Version 0.25, matching the recorded replay environment; Accessed 18 August 2026.
- [32] Qwen Team. Qwen2.5-1.5B-Instruct-GGUF model card, 2024. URL <https://huggingface.co/Qwen/Qwen2.5-1.5B-Instruct-GGUF>. Official model card documenting the GGUF distribution and Q4_K_M quantisation; Accessed 18 August 2026.
- [33] Hamid Rezatofighi, Nathan Tsoi, JunYoung Gwak, Amir Sadeghian, Ian Reid, and Silvio Savarese. Generalized intersection over union: A metric and a loss for bounding box regression. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 658–666, 2019. doi: 10.1109/CVPR.2019.00075. URL https://openaccess.thecvf.com/content_CVPR_2019/html/Rezatofighi_Generalized_Intersection_Over_Union_A_Metric_and_a_Loss_for_CVPR_2019_paper.html. Accessed 18 August 2026.
- [34] Robot State Publisher contributors. Robot State Publisher: ROS 2 Jazzy documentation, 2026. URL https://docs.ros.org/en/jazzy/p/robot_state_publisher/. Version 3.3.4; Accessed 18 August 2026.
- [35] ROS-Industrial Consortium. `staubli_tx2_60_support`: Stäubli TX2-60/TX2-60L robot support package, 2021. URL https://github.com/ros-industrial/staubli_experimental/tree/256a71acea2161b64e12af1a6b925a7f26695007/staubli_tx2_60_support. Revision 256a71a; Accessed 18 August 2026.
- [36] ROS-Industrial Consortium. `staubli_tx60_support`: Stäubli TX60 shared robot description assets, 2021. URL https://github.com/ros-industrial/staubli_experimental/tree/256a71acea2161b64e12af1a6b925a7f26695007/staubli_tx60_support. Revision 256a71a; Accessed 18 August 2026.
- [37] ROS-Industrial Consortium. `kuka_kr6_support`: KUKA KR 6 robot support package, 2025. URL https://github.com/ros-industrial/kuka_experimental/tree/8d9292b04a22628b1b78d989e2ddd3abb913bf92/kuka_kr6_support. Revision 8d9292b; Accessed 18 August 2026.
- [38] ROS-Industrial Consortium. `motoman_hc10_support`: Yaskawa Motoman HC10 and HC10DT robot support package, 2025. URL https://github.com/ros-industrial/motoman/tree/846fbbc57c60223f12bf4795ea6404982d57709e/motoman_hc10_support. Revision 846fbbc; Accessed 18 August 2026.
- [39] Tiago-Harmonic and PAL Robotics contributors. `tiago_description`: TIAGo robot description, 2025. URL https://github.com/Tiago-Harmonic/tiago_robot/tree/bc2ac55a0be2c49ccca6e09f6ff533902c9fb8e2/tiago_description. Tiago-Harmonic fork of PAL Robotics' `tiago_robot`, revision bc2ac55; Accessed 18 August 2026.
- [40] Université de Toulouse. Schéma de promotion des achats publics socialement et écologiquement responsables 2026–2029, 2026. URL https://www.univ-tlse3.fr/medias/fichier/2026-03-ca-083_1773392372102-pdf. Institutional figures on p. 5; definitive SPASER adopted by the university council on 9 March 2026; document dated 13 March 2026; Accessed 18 August 2026.
- [41] Université de Toulouse. La Maison de la Formation Jacqueline Auriol (MFJA), n.d. URL <https://www.univ-tlse3.fr/lieux-de-ressources/la-maison-de-la-formation-jacqueline-auriol-mfja>. Undated institutional page; Accessed 18 August 2026.
- [42] Université d'Évry Paris-Saclay. Évaluation de la formation en entreprise (Module Stage M2): Le rapport. Internal university guidance, 2026. Official internship-report instructions, June 2026, p. 17, Section V: use of generative artificial intelligence.
- [43] Junjie Wen, Yichen Zhu, Jinming Li, Minjie Zhu, Zhibin Tang, Kun Wu, Zhiyuan Xu, Ning Liu, Ran Cheng, Chaomin Shen, Yaxin Peng, Feifei Feng, and Jian Tang. TinyVLA: Toward fast, data-efficient vision-language-action models for robotic manipulation. *IEEE Robotics and Automation Letters*, 10(4):3988–3995, 2025. doi: 10.1109/LRA.2025.3544909. URL <https://doi.org/10.1109/LRA.2025.3544909>. Accessed 18 August 2026.
- [44] An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, et al. Qwen2.5 Technical Report. *arXiv preprint arXiv:2412.15115*, 2024. doi: 10.48550/arXiv.2412.15115. URL <https://arxiv.org/abs/2412.15115>. Accessed 18 August 2026.
- [45] Brianna Zitkovich, Tianhe Yu, Sichun Xu, Peng Xu, Ted Xiao, Fei Xia, Jialin Wu, Paul Wohlhart, Stefan Welker, Ayzaan Wahid, et al. RT-2: Vision-language-action models transfer web knowledge to robotic control. In *Proceedings*

of the 7th Conference on Robot Learning, volume 229 of *Proceedings of Machine Learning Research*, pages 2165–2183. PMLR, 2023. URL <https://proceedings.mlr.press/v229/zitkovich23a.html>. Accessed 18 August 2026.

Chapter A

Reproducibility Records

This appendix identifies the source, external artefacts and shortest paths to the records supporting chapter 11. Ubuntu 24.04, ROS 2 Jazzy, Gazebo Harmonic and Python 3.12 form the reference environment. The source repository is identified by [1]; build and operating procedures remain in `docs/INSTALLATION.md`, `runbook.html` and `docs/QUICK_START_AND_FEATURE_GUIDE.md`.

A.1 Identities and releases

Table A.1: Principal source, data and evidence identities.

Item	Identity and scope
Source and lightweight records	The report revision in the project repository contains the implementation, tests and compact evidence. The package is <code>report/evidence/room315_visual_v4_submission_2026-08-11</code> ; its <code>evidence_manifest.json</code> SHA-256 is <code>840787b617e0f671628dfc7d8122ff559d76664506ff8f94ac31d043737da446</code> , and its package-wide SHA256SUMS SHA-256 is <code>dec74565a8ccfba57a32d83dfd03236daeafeff226d5243594e328fa4adc5ab</code> .
Project demonstration videos	Online demonstration-video folder . The folder provides a visual overview of the project. These videos are supplementary; the frozen records and hashes in this appendix remain the evidence for the reported measurements.
Full runtime evidence	Release <code>room315-visual-v4-evidence-2026-08-11</code> targets <code>0d19e1601d57416b83c871c1a8d413ec0dd523a6</code> . Its 247,825,706-byte archive has SHA-256 <code>35b583baca4f45eed6aad659c253180d00f1a5830ce389266ba714a4445a8ecc</code> and contains the checkpoint, sealed Final Test and 18 ROS-bag recordings stored in the MCAP container format [9].
Data and replay	Release <code>v4-seed31520260811-dataset-v1</code> targets <code>a26dbd5569b5bb1ab4f794f96fbbd3e8486aca30</code> . It contains 5,528 Training, 512 Validation, 256 post-selection-regression and 1,040 Final-Test paired-camera scenes, both required checkpoints, frozen evaluation source and stateless replay.
Selected visual model	Epoch-11 checkpoint SHA-256: <code>869d64049b0092c37d21a4c8b910dc6b91954527e0e49c5694fa82dce570f40d</code> .
Final Test	The 1,040-scene dataset fingerprint is <code>226f4b5889f7d8b66bccc87d3e8dd494f710c8f4b228fab4851d6da7448e2eef</code> ; the run contains 2,080 images and 4,680 visible targets.

Large checkpoints, images and ROS bags remain outside Git because of size; the table’s hashes bind the exact objects used by the reported runs. A sealed dataset is frozen and hash-identified, not encrypted or still unseen. Public release enables replay without changing the historical role of a partition.

A.2 Single verification procedure

For either release, retrieve the assets using its retained README, confirm that the immutable tag resolves to the commit listed in table A.1, compare the downloaded archive with its declared SHA-256, then extract it and run the included package-wide `sha256sum --strict -c SHA256SUMS`. For the lightweight repository package, also compare the two manifest identities in the first table row. Any mismatch invalidates the provenance chain and the associated result claim. The release README files retain the platform-specific download, extraction and stateless replay commands so that those listings need not be duplicated here.

The replay re-evaluates the selected checkpoint but does not reopen the historical one-shot Final Test or claim bit-identical retraining across arbitrary hardware. The reference environment for the recorded replay used CUDA 12.8 [20] and is documented in `report/evidence/room315_visual_v4_dataset_release_v1/ENVIRONMENT.md`.

A.3 Result-to-record index

Paths beginning with `model/`, `final_test/` or `runtime/` are relative to the lightweight evidence package identified above.

Table A.2: Compact index of records supporting the reported results.

Result group	Primary record
Visual model and Final Test	<code>model/</code> stores training/selection controls; the historical <code>canary/</code> directory—a legacy path name for the post-selection regression records—stores that development check. <code>final_test/final/controls/</code> and <code>final_test/final/results/</code> store the sealed contract, completion ledger and measurements.
Observation and closed-loop campaigns	<code>runtime/shadow/</code> , <code>runtime/campaign/</code> and <code>runtime/safety/</code> store observation checks; <code>runtime/closed_loop/positive/</code> , <code>runtime/closed_loop/fault/</code> and <code>runtime/closed_loop/active_runtime/</code> store the positive, fault and selected Gazebo-runtime records.
Language interface	<code>report/evidence/room315_language_semantic_evaluation_v3</code> stores the fixed ten-case corpus, raw envelopes, fusion traces, environment and checksums.
Software and full-floor checks	The three <code>pytest_current_source*_2026-08-07.log</code> records and <code>report/evidence/multi_robot_cold_start_smoke_2026-08-07</code> support the named suites and six-robot launch result.
Runtime configuration and camera figure	Active files are under <code>mfja_robot_control_config/config/room_315_v1a/</code> ; execution defaults to false. B03 frame provenance and checksums are under <code>report/evidence/room315_v4_closed_loop_campaign_figures_2026-08-12</code> .

Exact per-file hashes and JSON pointers remain in `report/evidence/ACTIVE_EVIDENCE.md`; raw records preserve their original archival provenance. The index identifies where evidence can be inspected.

Chapter B

Technical Reference and Detailed Matrices

This appendix retains implementation and experiment detail that would interrupt the main argument. The chapters explain why each design was selected and what the results mean; the tables below preserve the corresponding configuration and case-level reference. Installed configuration, the runbook and the hash-identified records in appendix A remain authoritative.

B.1 Robot and rail configuration

The following table consolidates package ownership, rail geometry, device semantics and maintenance constraints used by the integrated platform.

Table B.1: Operational configuration moved from the main chapters.

Area	Reference detail
Package ownership	<code>mfja_3rd_floor_description</code> owns worlds and models; <code>mfja_3rd_floor_bringup</code> composes them; <code>mfja_rail_interfaces</code> owns public rail types; and <code>mfja_robot_control_config</code> owns controllers, planning, perception, configuration and tests.
Robot helper	<code>robot_joint_command.py</code> validates the selected profile, joint-vector length and angular/linear units before publishing to the instance namespace. Exact invocations remain in <code>runbook.html</code> .
Rail source and mapping	Fourteen CSV curves contain 276 ordered <code>index, x, y, z</code> centre-line rows. Twelve named anchors join curve ends; side-specific affine scale/rotation/translation and height/yaw offsets map the source geometry into Gazebo.
Devices	Switches retain inherited IDs A1–A4. DZI names identify indexed slot detectors; DA names identify switch approaches/branches. Stopper state 0 is released and 1 held; linked detectors are configured 0.10 m upstream, and the switch protected distance is 0.35 m.
Fleet lifecycle	Per-shuttle commands are ON, OFF, RESET and REMOVE; ON/OFF/ RESET also accept ALL. Reported controller modes are MOVING, WAITING, DISABLED and the latched no-successor mode FALLING.
Motion and separation	Controller and pose-publication rate limits default to 30 Hz; default minimum centre separation is 0.33 m. The stored speed field is a setting, not an instantaneous-velocity measurement.
Maintenance and staging	The device-position tool defaults to a non-writing projection preview and guards excessive residuals. A retained clearance certificate binds shuttle identity, interior destination, entry sensor, bounded motion, controller stop and post-stop observation; later ON motion invalidates it.

B.2 Contracts and symbolic decisions

The following table identifies the principal records exchanged across the closed loop and the components authorised to produce them.

Table B.2: Principal closed-loop contracts and authorised producers.

Contract	Meaning	Allowed producer
ObservedFact/ ObservedState	Values with known/unknown/stale/conflicting status, freshness and provenance	Authorised visual/rule provider, then state-fusion layer
SemanticParseEnvelope	Strict semantic proposal; PDDL, plan and command fields are forbidden	Local language model
TaskGoalDraft/TaskGoal	Fused proposal followed by immutable, domain-valid and optionally grounded constraints	Field resolver, domain validator and grounding gateway
PddlPlanStep/ TranslatedPlanStep	Returned symbolic action, then deterministic sparse command metadata	PlanSys2 boundary, then plan translator
Checked command and typed ROS 2 messages	Authorised named actuation intent	Executive/macro dispatcher, then rule-based supervisor
PostconditionCheck and task records	Observed effect, reason and step/task outcome	Executive using controller, sensor and observation reports

B.3 Visual-model lineage

The predecessor V3 grouped split contained 1,528 training scenes and 256 scenes each for Validation and the consumed historical Test. The V3 Validation-selected backbone initialised V4, but neither historical evaluation partition enters V4. The active V4 output contains, for each of eight known identities, 14 segment scores, two loaded-state scores, four normalised box values and one within-segment ratio: $8(14 + 2 + 4 + 1) = 168$ learned values. A derived 200-value adapter is diagnostic only. The selected optimisation, sampling and evaluation controls now appear beside their rationale in table 9.3; online and offline thresholds are distinguished in table 11.2.

B.4 Detailed experiment matrices

The following matrices distinguish closed-loop case coverage, per-rail Final Test metrics, observation-only checks and symbolic-runtime boundaries.

Table B.3: Case-level detail for the closed-loop campaign.

Case	Family	Rail/shuttle	Recorded task path	Result
B01	Colour alias	Right/R1	Slot 1 to slot 2	1/1 passed
B02	Explicit identity	Right/R2	Slot 2 to slot 3	1/1 passed
B03	Colour alias	Right/R3	Slot 3 to slot 4	1/1 passed
B04	Station target	Right/R4	Slot 4 to Yaskawa slot 1	1/1 passed
B05	Colour alias	Left/L1	Slot 1 to slot 2	1/1 passed
B06	Explicit identity	Left/L2	Slot 2 to slot 3	1/1 passed
B07	Colour alias	Left/L3	Slot 3 to slot 4	1/1 passed
B08	Station target	Left/L4	Slot 4 to Yaskawa slot 1	1/1 passed
P01	Payload-nearest	Right/R1	R2 cleared; slot 1 to slot 3	4/4 passed
P02	Payload-nearest	Left/L1	L2 cleared; slot 1 to slot 3	4/4 passed
M01	Multi-shuttle route	Right/R1	R2 cleared; slot 1 to slot 3	4/4 passed
M02	Multi-shuttle route	Left/L1	L2 cleared; slot 1 to slot 3	4/4 passed

Table B.4: Per-rail metrics from the one-shot Final Test.

Metric	Global	Left	Right
Segment top-1	99.9359%	99.8726%	100.0000%
Segment macro F1	99.9117%	99.7460%	100.0000%
Loaded-state accuracy	99.9145%	99.9151%	99.9140%
Joint segment and $ \Delta s_{ratio} \leq 0.05$	90.8974%	89.8089%	92.0000%
Joint segment and $ \Delta s_{ratio} \leq 0.12$	99.0385%	98.9384%	99.1398%
Correct-segment s_m MAE	0.01573 m	0.01552 m	0.01594 m
Correct-segment s_m p95	0.05850 m	0.05660 m	0.06030 m
Mean box IoU	0.83400	0.83103	0.83701

Table B.5: Development and observation-only reference results.

Check	Recorded result	Interpretation
Validation / post-selection regression	76/76 and 74/74 gates passed; segment top-1 99.9113% and 100.0000%; joint 0.05 94.9867% and 94.5622%	Development evidence, not independent Test evidence
Same-frame shadow	55 paired frames; 52 accepted; no wrong-side identities or reported comparison errors	Remaining three rejections are not explained by the aggregate record
Seven-scene acceptance	7/7 scenes; 53/53 reobservations at 0.12; 150/169 identity checks at 0.05	Observation only; 19 strict misses repeat two biased A12E estimates
Visual-input faults / dense dry run	8/8 faults rejected; 10/10 dense-scene reobservations matched	PlanSys2 mutation and ac-tuation disabled

Table B.6: Files and boundaries of the implemented symbolic-planning path.

Boundary	Retained implementation detail
Domain	<code>domain_room315_runtime.pddl</code> defines the typed predicates and 21 action schemas used by the runtime
Problem construction	The scenario builder creates a fresh <code>:init</code> and goal from accepted state; unknown, stale or conflicting input prevents problem generation
Planner boundary	PlanSys2 sends the problem to POPF. Learned inputs provide checked goal and scene records but neither call the solver nor publish a command
Translation	A deterministic whitelist checks each returned action name, typed argument and sparse-command mapping before any command request exists
Supervised execution	The executive offers only the first supported action; the rule-based supervisor rechecks its command fields and current authority
Effect and replan	The executive compares the declared postconditions with observed reports and rebuilds a fresh problem before continuing